Generative AI from First Principles

Lecture 4: Transformers

Architecture, Computation, and Language Modeling

Lecture 4: Transformers

Previously: attention, through translation; a toy translator built from attention alone
Today: the Transformer, part by part, and the language model it became

1. **Building the Transformer**

   from Lecture 3's toy translator to the encoder–decoder of 2017, one part at a time

2. **The language model**

   the decoder alone, trained on children's stories; positions as rotations

3. **Strengths, limitations, and variants**

   every weight and its cost; the key–value cache; what it does well and where it falls short; variants

A Translator Built from Attention Alone

Lecture 3's toy: one read per target word, and nothing else

1. query: from the previous target word alone

$$\mathbf{q}_t = E_Q(y_{t-1})$$

2. read: score every source word, softmax, average the values

$$e_{tj} = \mathbf{q}_t \mathbf{k}_j^\top, \qquad \alpha_t = \mathrm{softmax}(e_{t1}, \dots, e_{tn}),$$
$$\mathbf{c}_t = \sum_j \alpha_{tj} \mathbf{v}_j$$

3. predict: from the previous word and the read

$$q(y_t \mid y_{<t}, \mathbf{x}) = \mathrm{softmax}([E(y_{t-1}); \mathbf{c}_t] U)$$

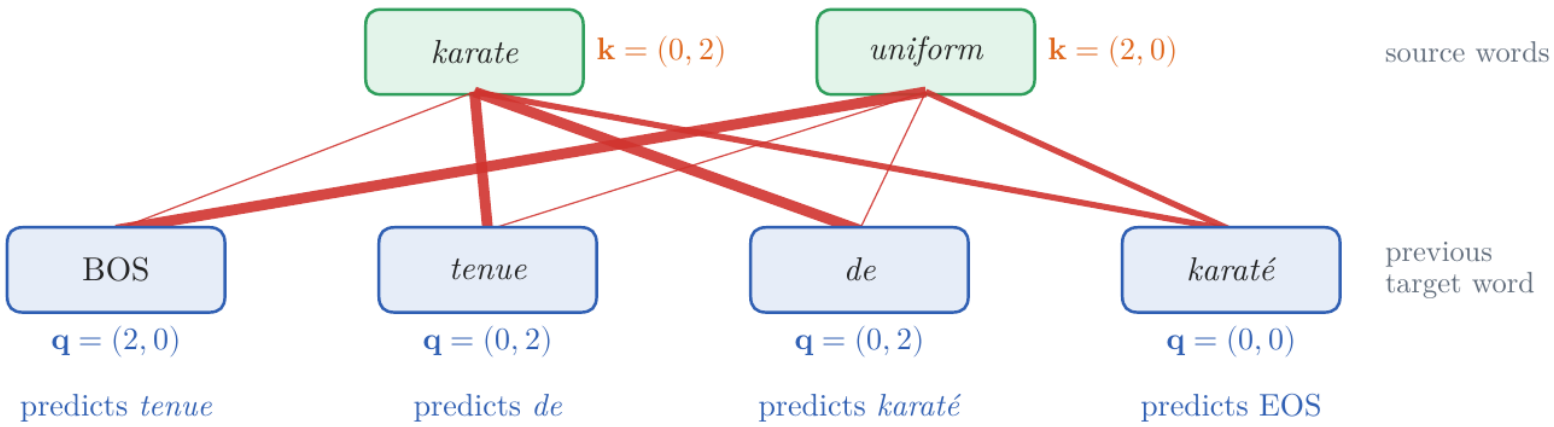


Figure 1: on karate uniform, each target position (bottom) reads the source words (top); line width is weight

A Translator Built from Attention Alone

Every translator of Part 1: the same data, training, and measures

the data: Multi30k, English captions of photographs with French translations; 29,000 pairs to train, 1,014 to validate, 1,000 to test

the training: 3,000 steps of 64 caption pairs; vectors of d = 192; Adam, with a warmup

two measures: cross-entropy on the validation captions (nats per French word); BLEU on the test captions, translated greedily

the bar: Lecture 3's LSTMs, 19.5 BLEU without attention and 49.2 with



a girl in karate uniform breaking a stick with a front kick .
une fille en tenue de karaté brisant un bâton avec un coup de pied .

Photograph: Flickr30k image 101362133 (Young et al., 2014), with its Multi30k caption

A Translator Built from Attention Alone

The toy, trained: 1.7 BLEU

the toy: cross-entropy 3.04, **1.7 BLEU** (the LSTM without attention, 19.5)

word order: the same words in another order give the same translation

a loop: once a word recurs, the same read, the same next word

why: nothing tells it where a word stands, or what it has written beyond the last word

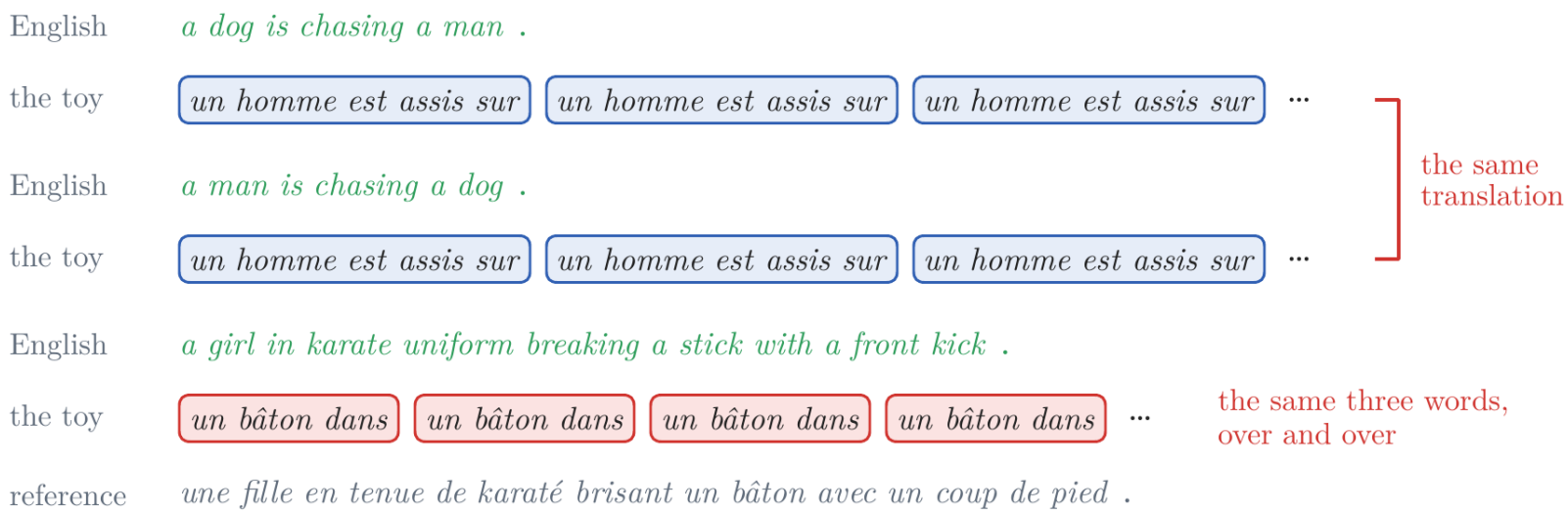


Figure 3: the toy's greedy output; the reference in gray

A Translator Built from Attention Alone

What the toy lacks, and the part of the Transformer that supplies it

the toy's read: the decoder's cross-attention (queries from the decoder, keys and values from the encoder)

1. where a word stands: word orders, the loop → **position vectors** (§2)

2. a word's neighbors: each source word read alone → **self-attention** (§3), a stack of blocks (§4)

3. what it has written: only the last word → **masked self-attention** (§3)

4. one kind of read, no computation → **several heads**, and an **MLP** at each position (§4)

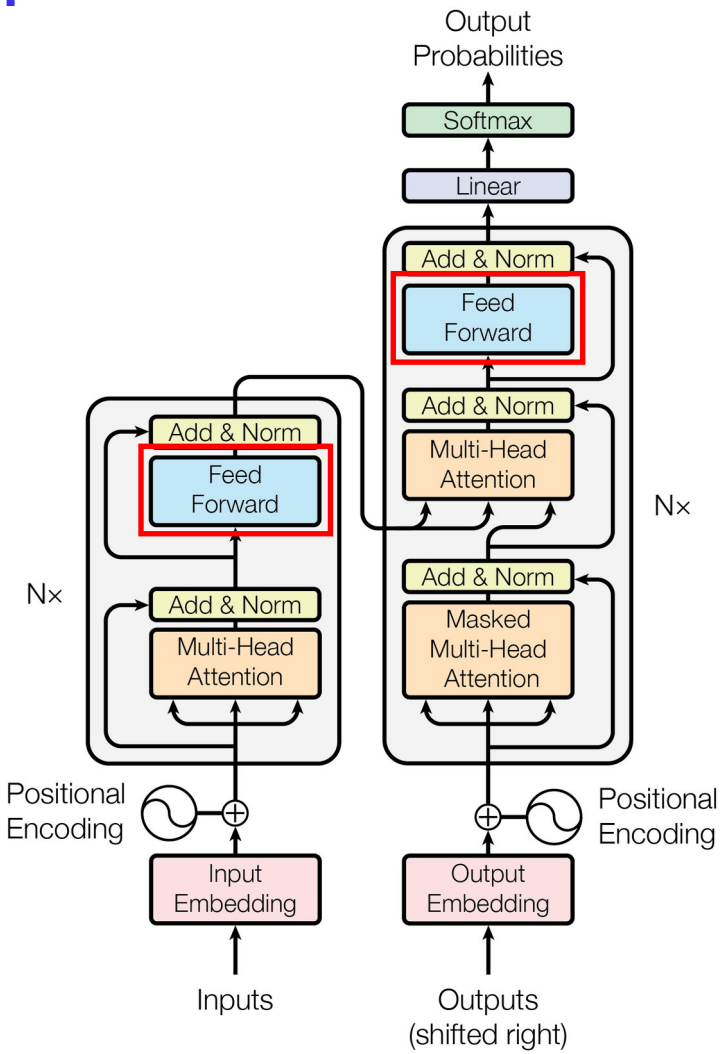


Figure 2: Vaswani et al. (2017), Fig. 1

Word Order: Positions

A vector for each word, and maps to queries, keys, and values

the problem: the reads see the source as a set; each query depends only on the previous word → **the fix here**: mark each position with a vector

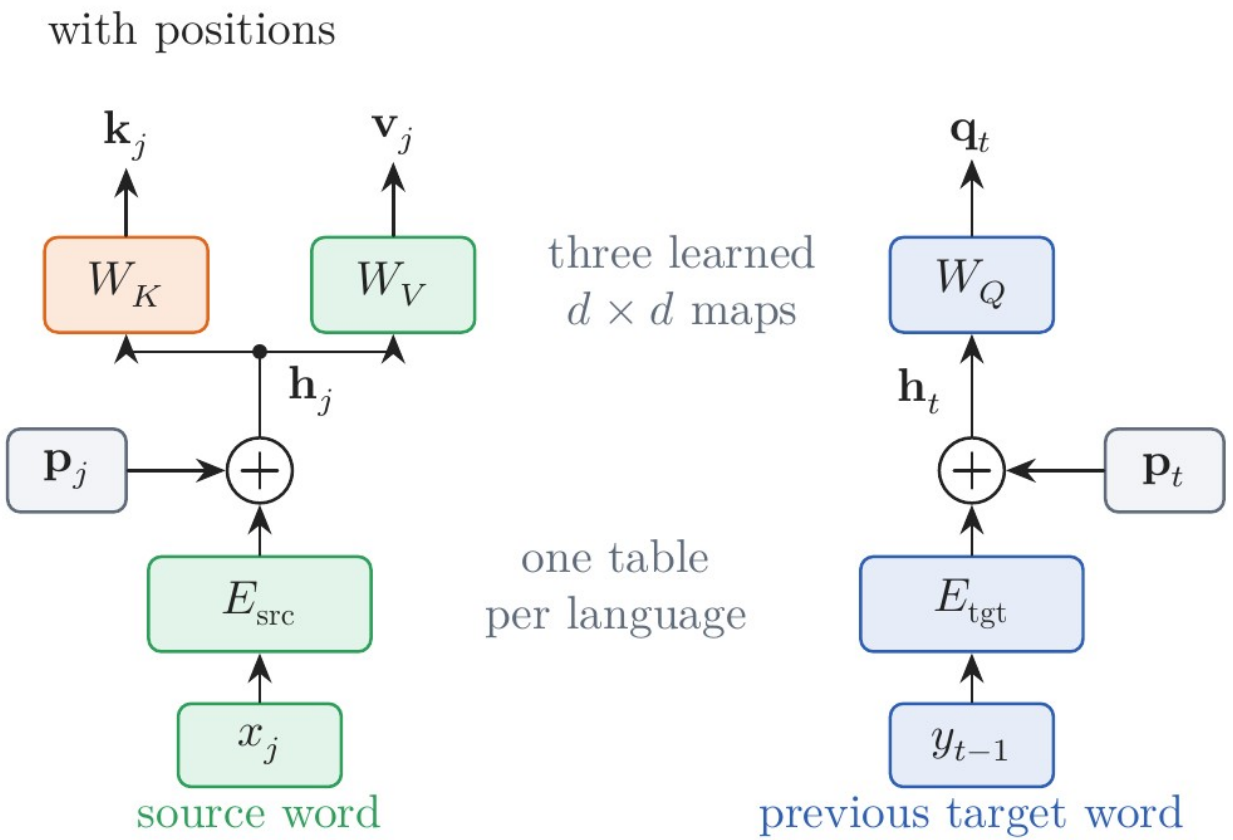
recall: a query says what a position looks for, a key what it is found by, a value what it passes on

one vector per word, plus its position's: one table per language

$$\mathbf{h}_j = E_{\text{src}}(x_j) + \mathbf{p}_j, \qquad \mathbf{h}_t = E_{\text{tgt}}(y_{t-1}) + \mathbf{p}_t$$

three learned maps: query, key, and value from that one vector

$$\mathbf{q}_t = \mathbf{h}_t W_Q, \qquad \mathbf{k}_j = \mathbf{h}_j W_K, \qquad \mathbf{v}_j = \mathbf{h}_j W_V$$



Word Order: Positions

What a position vector should be

distinct: the third word is not the fifth

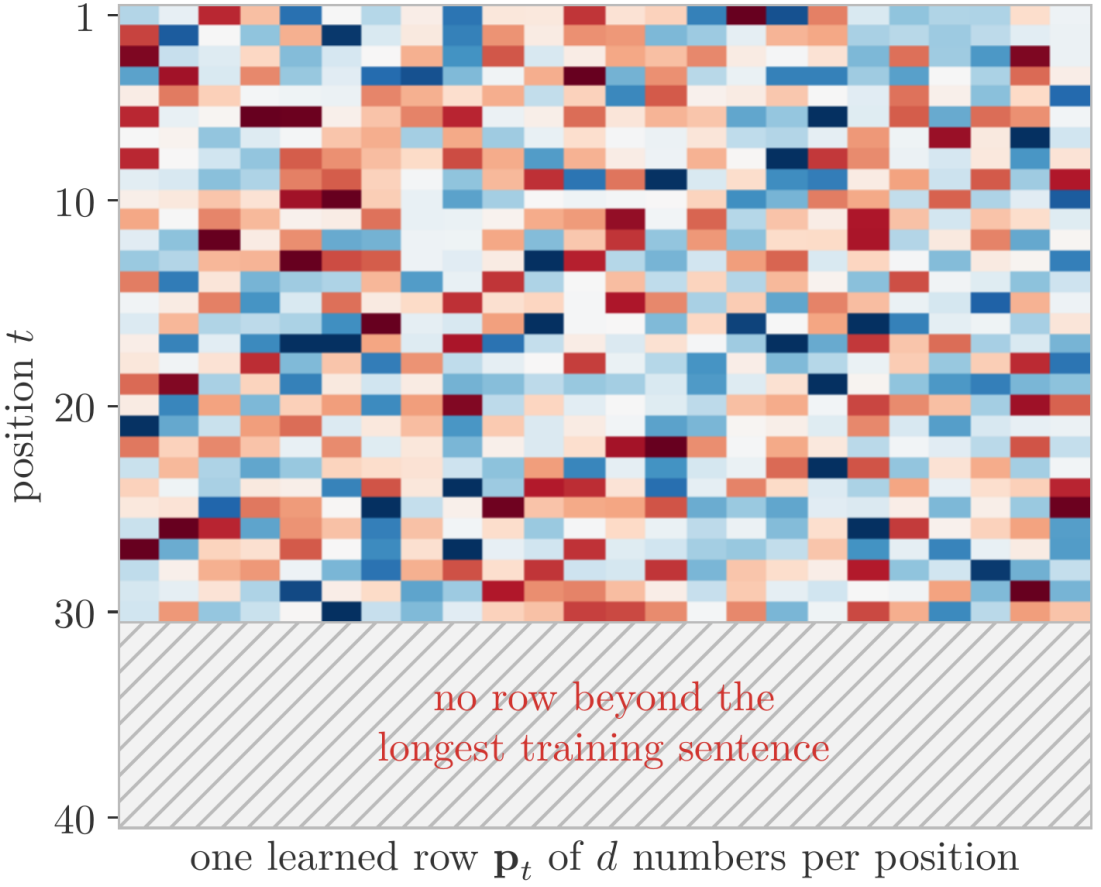
fixed: position 3 is the same in every sentence

bounded: it must not drown the word's vector

defined for every position: even beyond the longest training sentence

relative: t → t + u changes it the same way for every t, so “two words back” works everywhere

learned vectors, one row per position: the first three, **not the last two**



Slide figure: an illustrative learned table of position vectors

Word Order: Positions

Sinusoids: a clock with many hands

pairs of sines and cosines: each pair turns at its own rate as t grows

$$p_{t,2i} = \sin(t\,\omega_i), \qquad p_{t,2i+1} = \cos(t\,\omega_i),$$
$$\omega_i = 10000^{-2i/d}$$

an offset is a rotation: t → t + u turns every pair by u ω_i, the same for every t

$$\mathbf{p}_t \mathbf{p}_j^\top = \sum_i \cos((j-t)\,\omega_i)$$

why this design: the paper's two hopes, attending by relative position and reading longer sentences; §9 gives the deeper reason

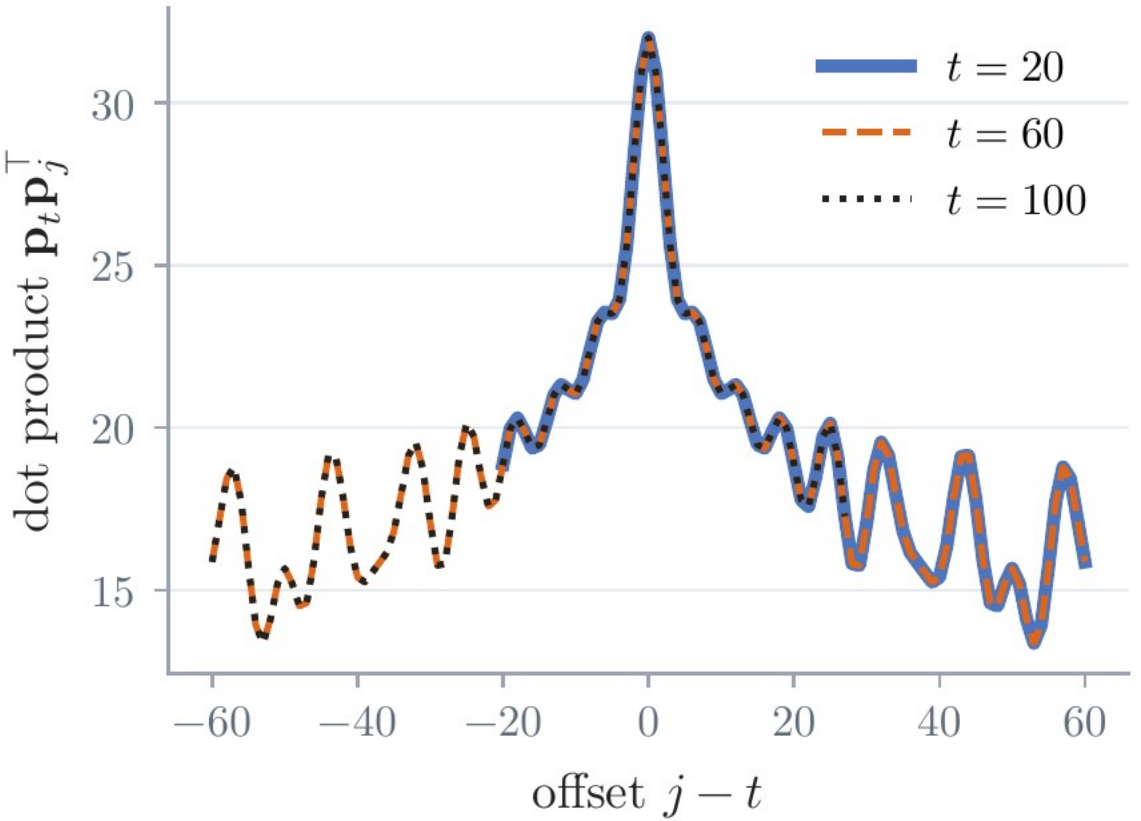


Figure 5b: the dot product depends only on the offset

Word Order: Positions

What positions change: 1.7 → 39.3 BLEU

the experiment: the toy, with positions and the learned maps, trained as before; cross-entropy 3.04 → 1.42, BLEU **1.7 → 39.3**

both failures gone: two orders, two translations; the karate caption no longer loops

two gaps remain: uniforme for uniform, word for word; and it stops before the kick (§3)

English	*a dog is chasing a man .*
with positions	*un chien court après un homme court .*
English	*a man is chasing a dog .*
with positions	*un homme court après un chien .*
English	*a girl in karate uniform breaking a stick with a front kick .*
with positions	*une fille en uniforme de karaté avec un bâton .*
reference	*une fille en tenue de karaté brisant un bâton avec un coup de pied .*

two orders, two translations

uniforme, word for word; stops before the kick

Figure 6: greedy output with positions; the reference in gray

Word Order: Positions

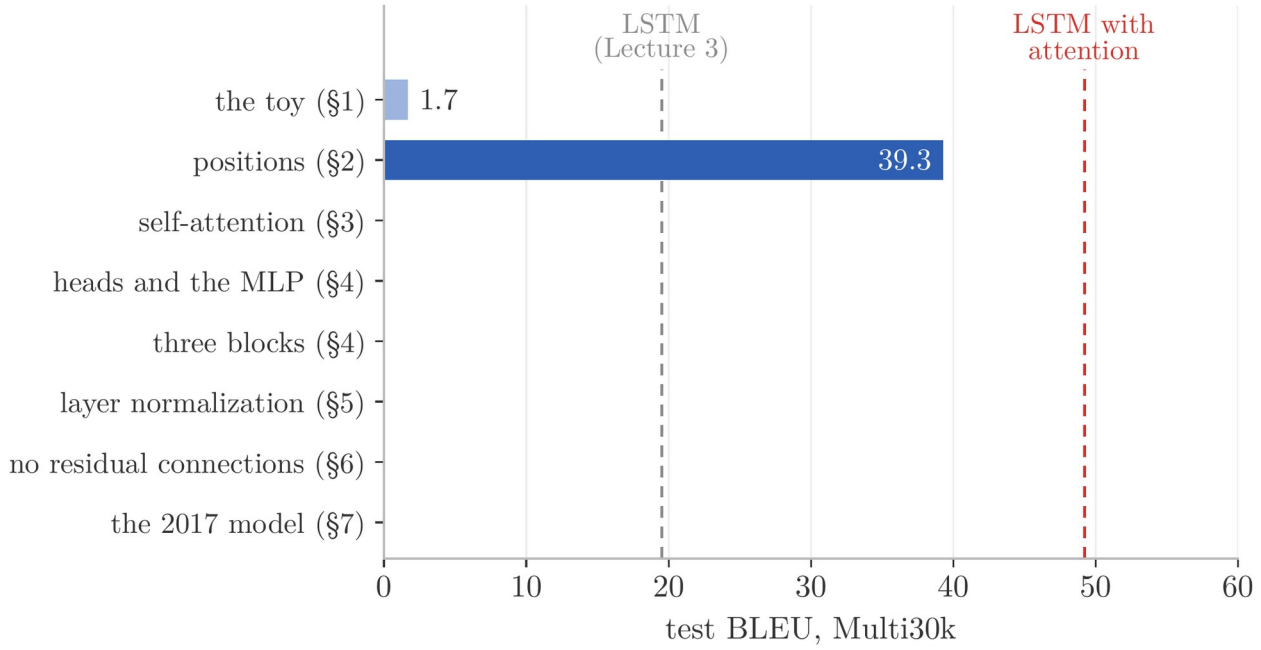
What positions change: the reads

the toy's reads: rows after the same French word are identical

with positions: each French position has its own query; the reads follow the caption

learned or sinusoidal: learned 42.9 against 39.3 here (one run); 25.7 against 25.8 in the paper

the build so far: the largest step of the lecture



Context: Self-Attention

Reading one's own sentence

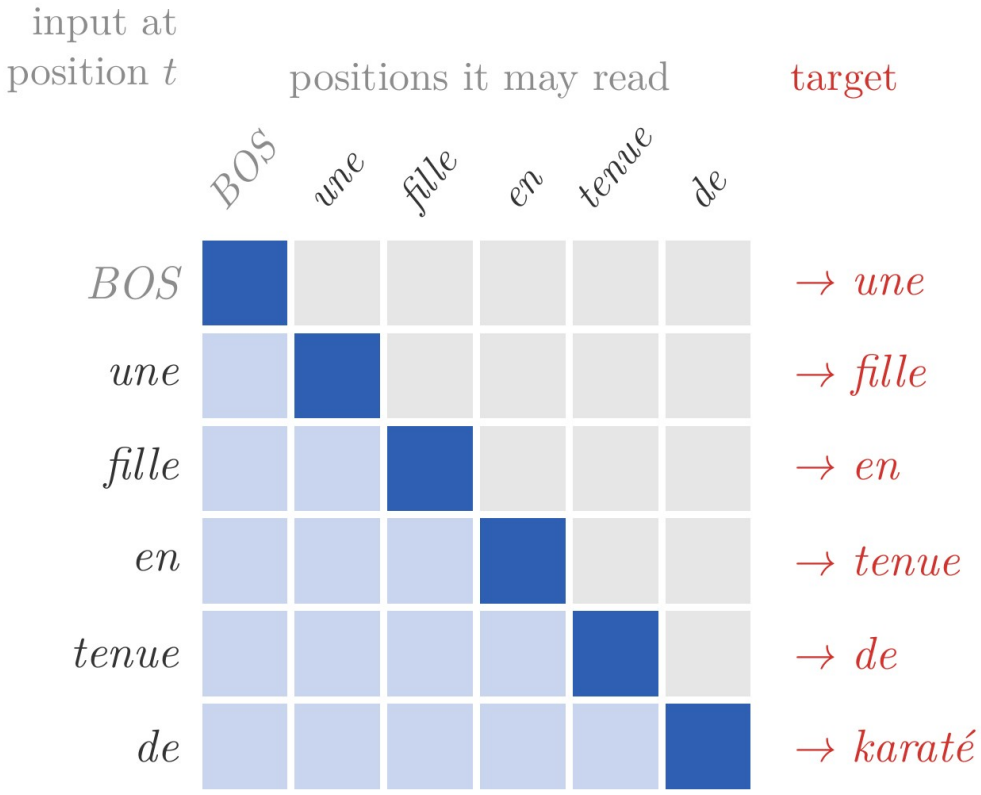
the problem: *uniforme* for *uniform*, since each source word is read alone; each French position knows only its place and the last word → **the fix here**: the positions of one sentence read one another

a sentence: T positions, the rows $h_1, \ldots, h_T$ of a $T \times d$ matrix X

in the encoder: every English position may read every other

in the decoder: position t holds y_{t-1} and predicts y_t, so it may read only $j \le t$, the translation so far

each read: c_t, a weighted average of the vectors t may read; how should the weights be chosen?



Slide figure: the decoder; each row reads the blue squares (dark, its own input)

Context: Self-Attention

Choosing the weights

equal weights: c_t averages h_1, ..., h_t

as matrices: a mask of $-\infty$ above the diagonal

$$M_{tj} = \begin{cases} 0 & j \le t \\ -\infty & j > t \end{cases}$$

$$A = \mathrm{softmax}(M), \qquad C = AX$$

weights from similarity: dot products of the vectors

$$A = \mathrm{softmax}(XX^\top + M)$$

each word mostly reads itself: 0.82 of its weight; and a pronoun looks for its noun, not the reverse

the remedy of §2: learned queries, keys, and values

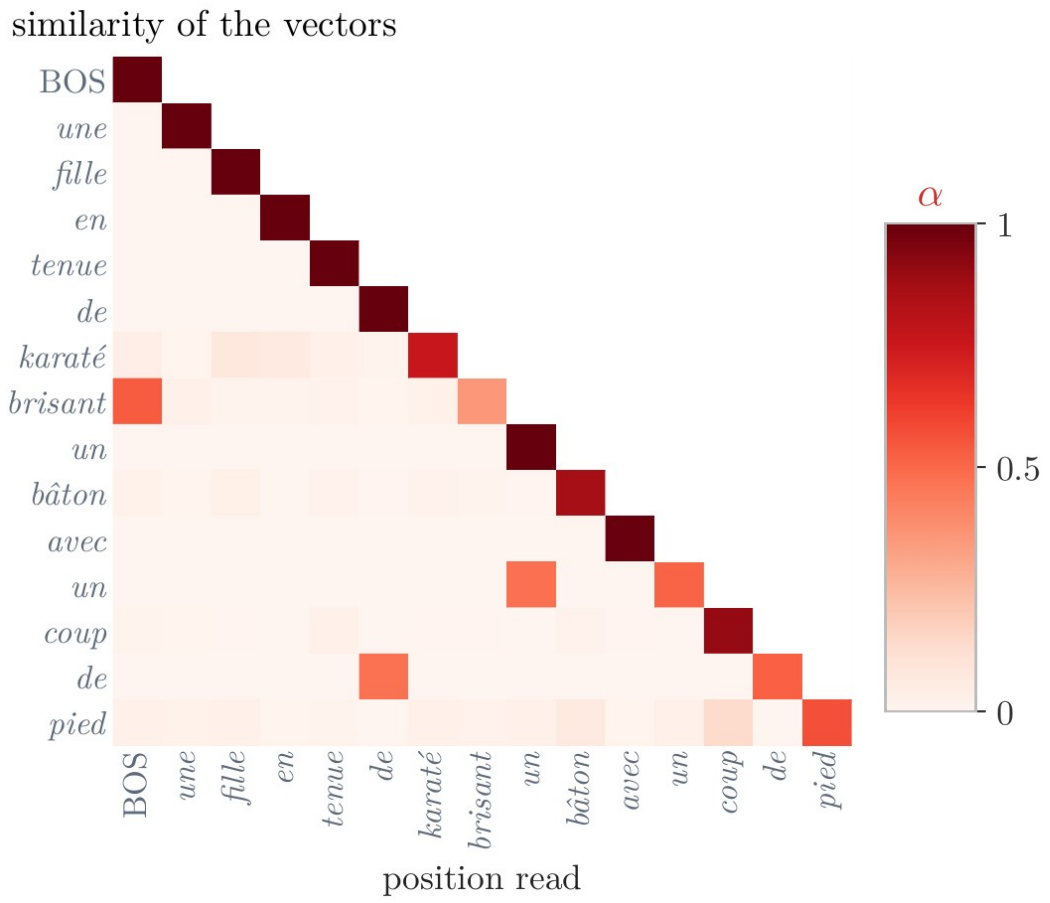


Figure 8b: dot products of the vectors; each row sums to 1

Context: Self-Attention

Self-attention: queries, keys, and values from one sentence

three maps of the same sentence

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

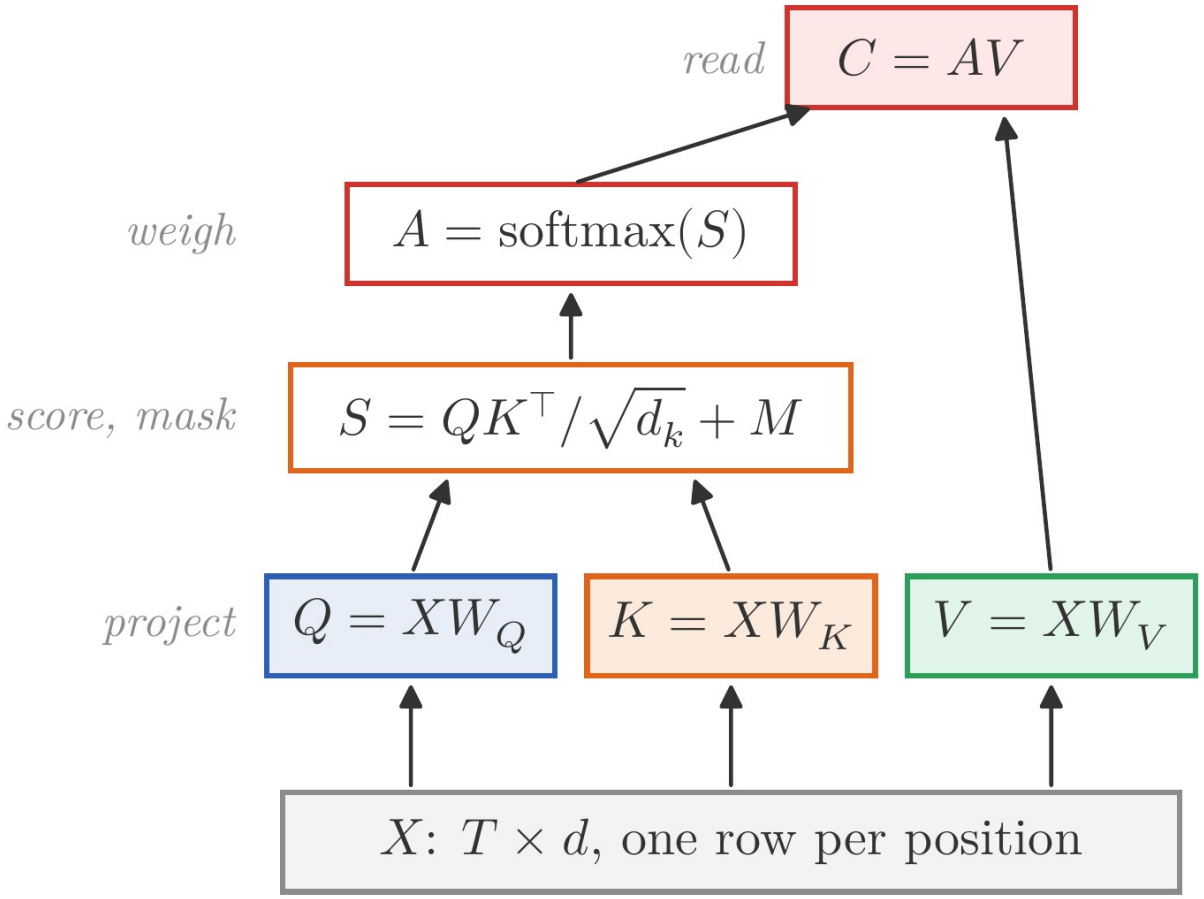
the read: softmax of the scaled scores, masked, times the values

$$C = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

not symmetric: $W_Q W_K^\top$ need not be, so a pronoun can look for its noun, not the reverse

$$e_{tj} = \frac{\mathbf{q}_t \mathbf{k}_j^\top}{\sqrt{d_k}} = \frac{\mathbf{h}_t W_Q W_K^\top \mathbf{h}_j^\top}{\sqrt{d_k}}$$

found by one thing, passes on another: the value need not be the key



Slide figure: one head of self-attention, step by step

Context: Self-Attention

One head by hand: lily saw tom . she

hand-chosen maps: a key says (is a name, is female), a query (wants a name, wants a female), a value (is a character, lily rather than tom)

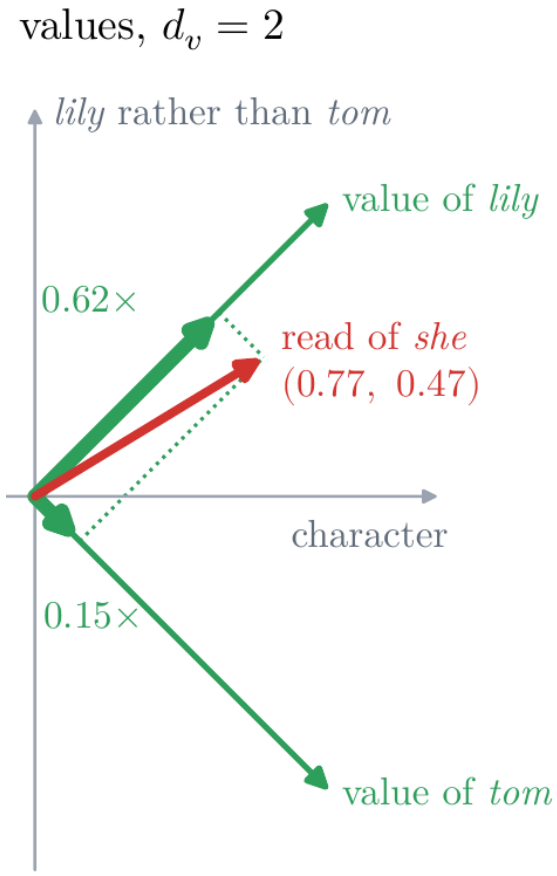
	lily	*saw*	*tom*	.	*she*
query	$(0,0)$	$(2,0)$	$(0,0)$	$(0,0)$	$(2,2)$
key	$(1,1)$	$(0,0)$	$(1,0)$	$(0,0)$	$(0,1)$
value	$(1,1)$	$(0,0)$	$(1,-1)$	$(0,0)$	$(0,0)$

she scores the keys: lily's key reaches furthest along its query

$$\alpha_{she} = (0.62,\ 0.04,\ 0.15,\ 0.04,\ 0.15)$$

the read: a weighted sum of the values, about lily, by content, not position

$$\mathbf{c}_{she} = 0.62\,(1,1) + 0.15\,(1,-1) = (0.77,\ 0.47)$$



Context: Self-Attention

The matrix form: one head in five steps

1. project: Q and K are T × d_k, V is T × d_v

2. score: S = $QK^T/\sqrt{d_k}$, which is T × T

3. mask: $-\infty$ above the diagonal

4. weigh: A, the softmax of each row

5. read: C = AV, which is T × d_v

all matrix products: only steps 2 and 5 mix positions; parallel hardware does them fastest

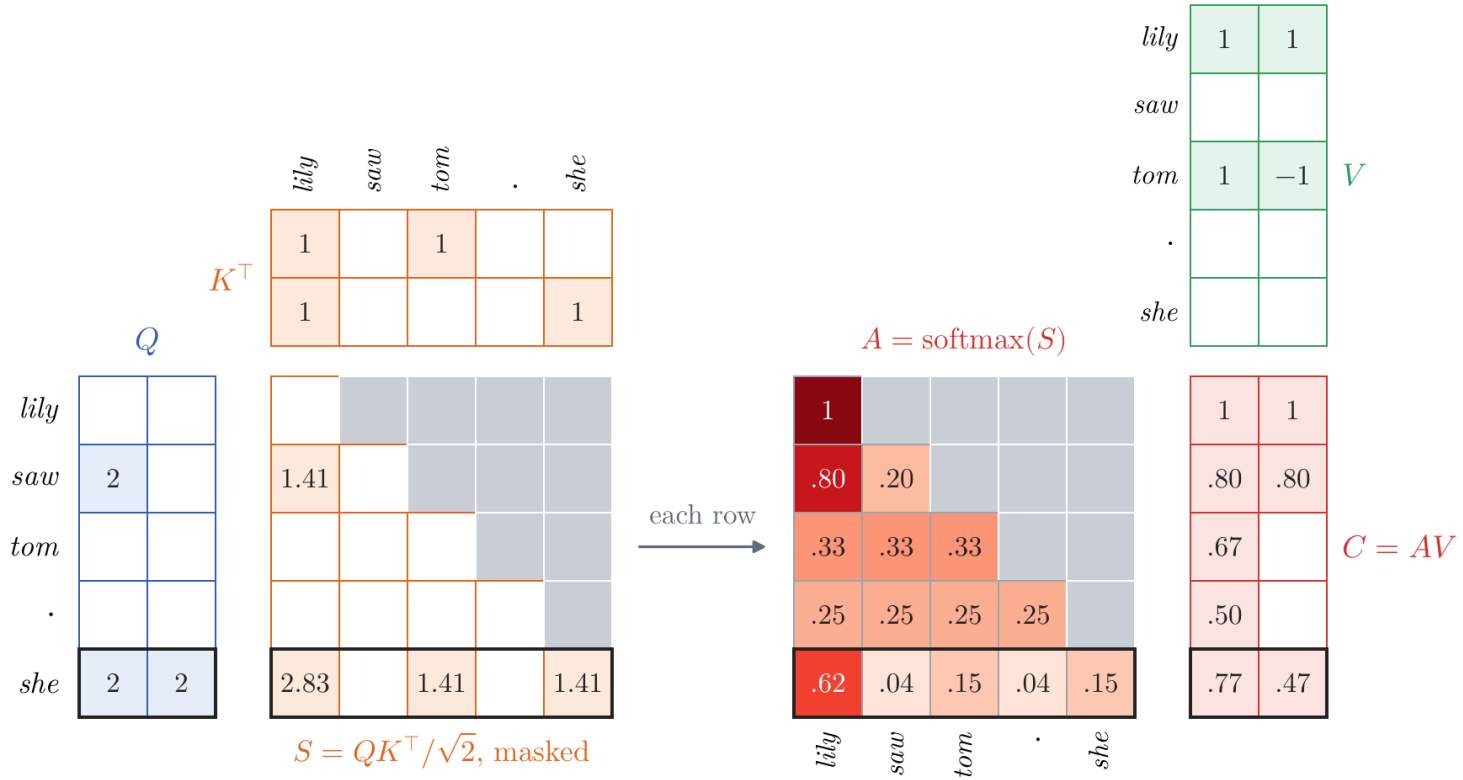


Figure 10: the example above; the row of she outlined

Context: Self-Attention

Why divide by $\sqrt{d_k}$

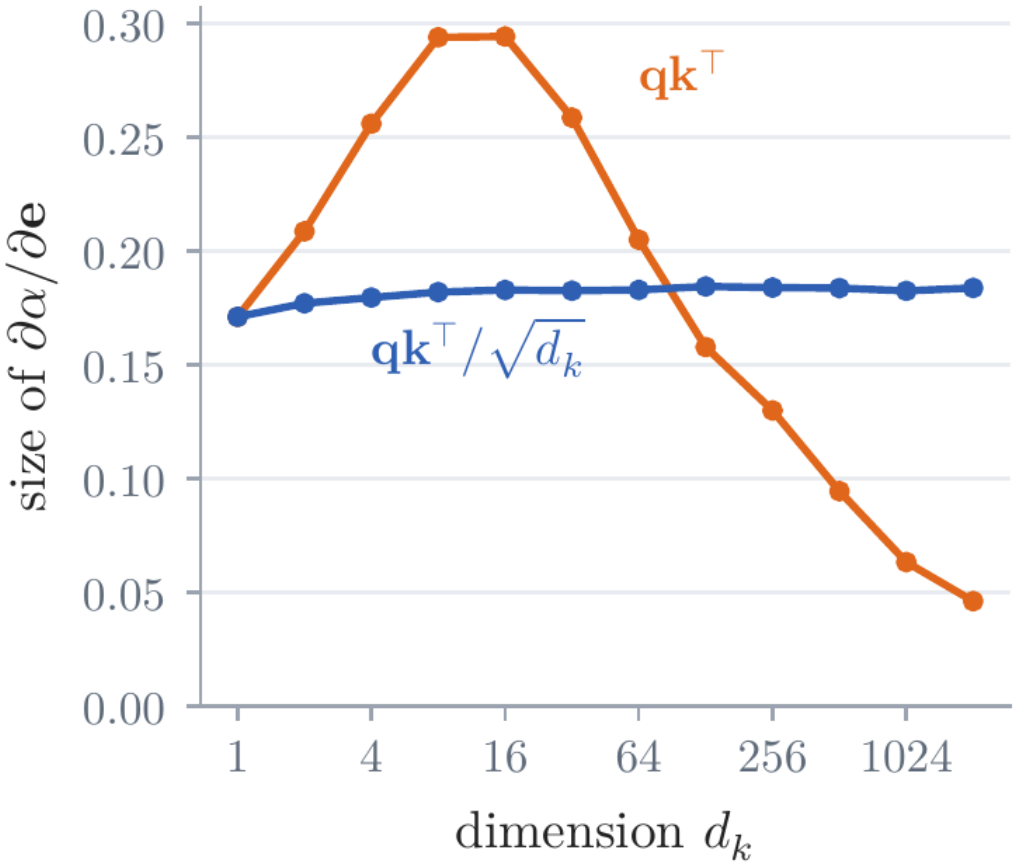
the experiment: random queries and keys, as at initialization; one query, 64 keys, 2,000 trials

the dot product has variance d_k: with $d_k = 256$ the scores spread by 16; almost 90% of the weight on one key

$$\mathrm{Var}(\mathbf{q}\mathbf{k}^\top) = \mathrm{Var}\left(\sum_{i=1}^{d_k} q_i k_i\right) = d_k$$

one weight near 1: the softmax's derivatives near zero, so no gradient

divide by $\sqrt{d_k}$: variance 1 for every d_k, Lecture 2's temperature



Context: Self-Attention

The causal mask: every prediction in one pass

teacher forcing: the input at position t is the reference's previous word, y_{t-1}

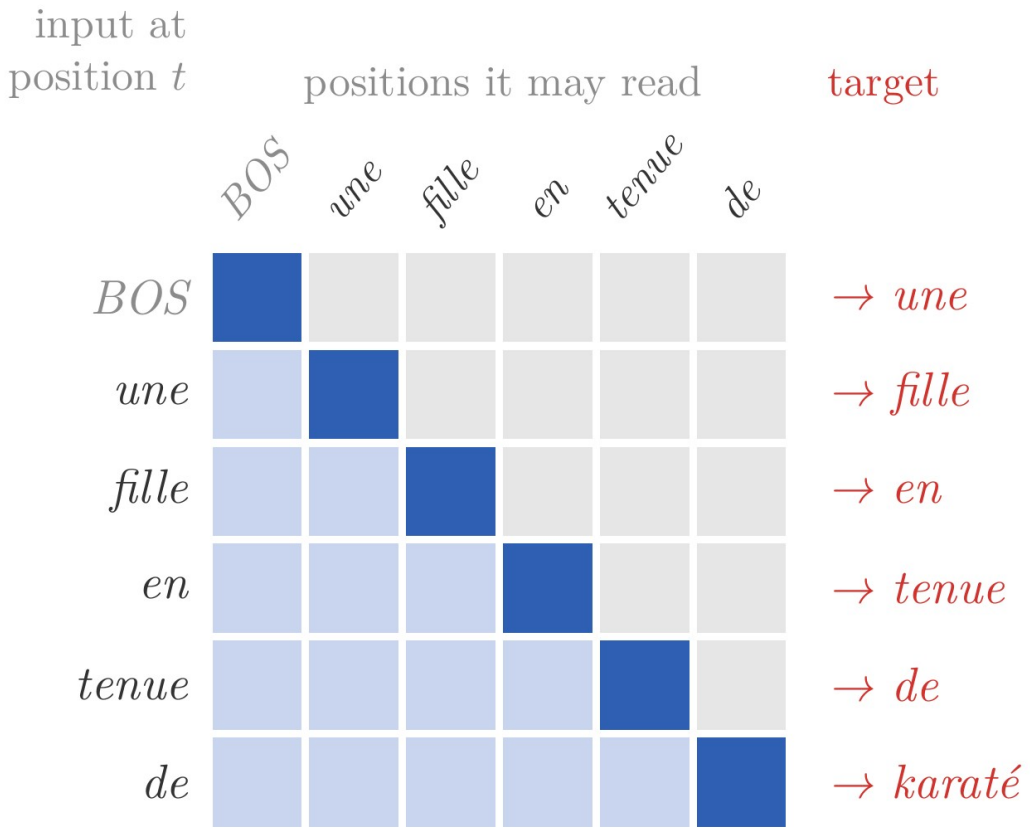
reading ahead is reading the answer: position t + 1 holds y_t

the mask is enough

1. within a position, q_t, k_t, v_t come from h_t
2. within a layer, c_t depends only on h_1, …, h_t
3. through the stack, everything else acts on one position or reads the source

one pass trains every prediction: a recurrent decoder needed one step per word

the encoder needs no mask: the whole English caption is known



Slide figure: the decoder in training; blue may be read (dark, its own input), gray is masked

Context: Self-Attention

Sentences of any length

T is not built into the model: W_Q, W_K, W_V act on one position at a time; only the T × T weights change size

source and target: T = n for the caption, m for BOS, y_1, ..., y_{m-1}; the read of the source is m × n

a batch: pad each caption to the batch's longest; a padding mask adds −∞ for the filler, and the loss ignores it

in generation: one word at a time; T grows by one with each word

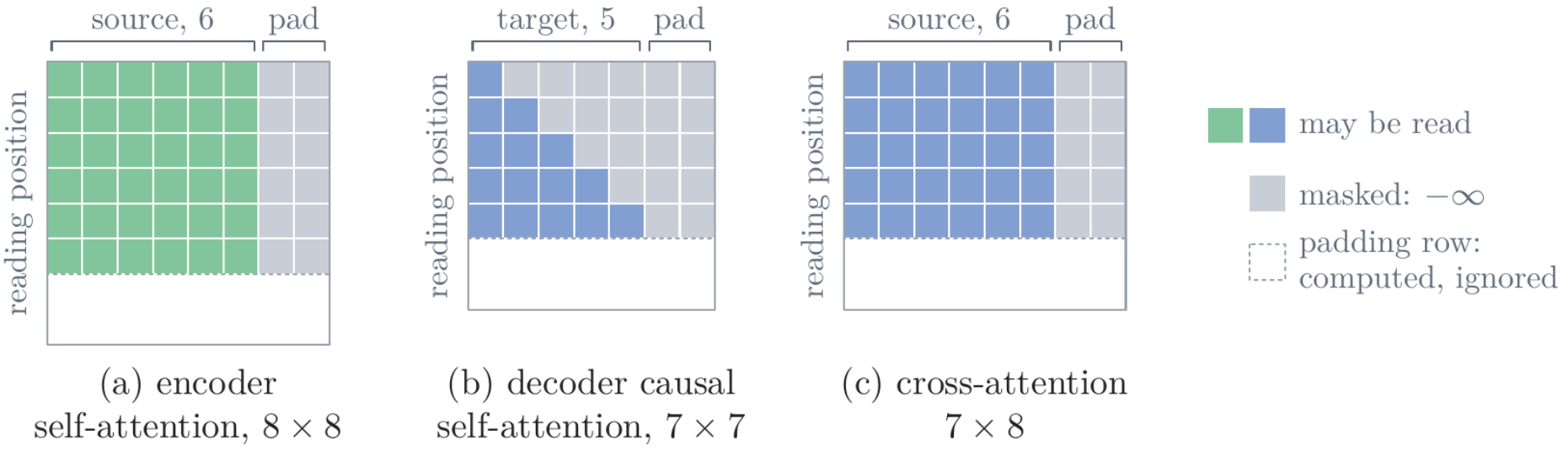


Figure 12: which positions (columns) each position (row) may read, for one padded caption pair

Context: Self-Attention

Adding the read to the vector

add, do not replace: each read is added to the vector it came from, a residual connection (§6)

$$\mathbf{h}_t \leftarrow \mathbf{h}_t + \mathbf{c}_t$$

the logits: from the decoder's last vector at t

$$q(y_t \mid y_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{h}_t U)$$

a word in context: bank moves toward river, or toward money

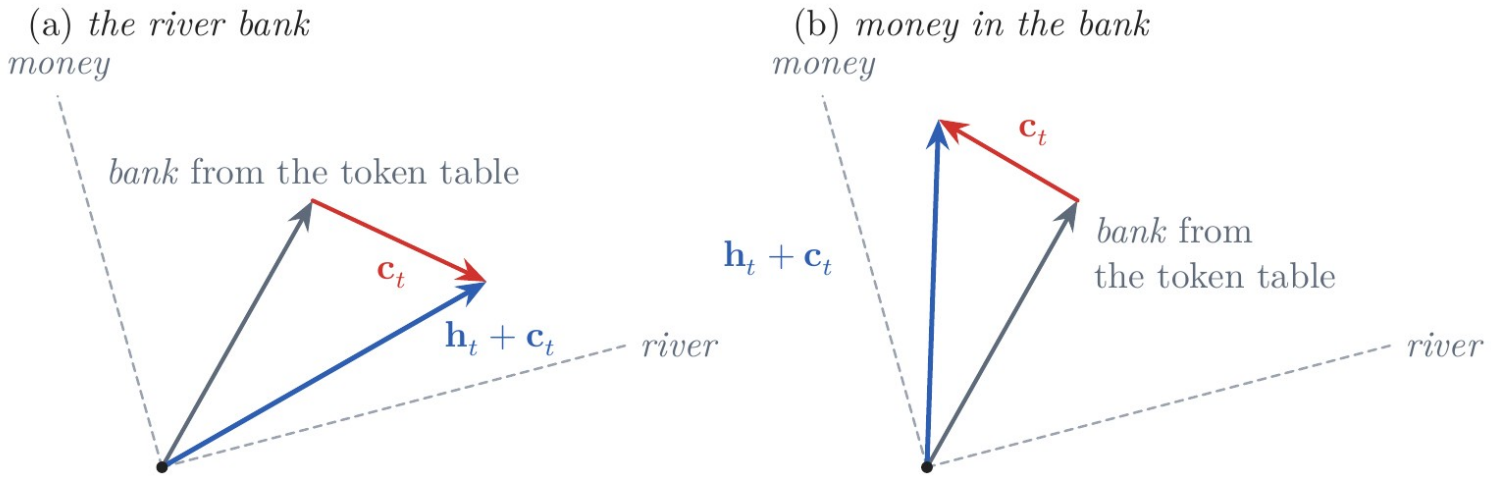


Figure 13: schematic, in two of the d directions; gray, bank from the token table; red, the read; blue, the sum

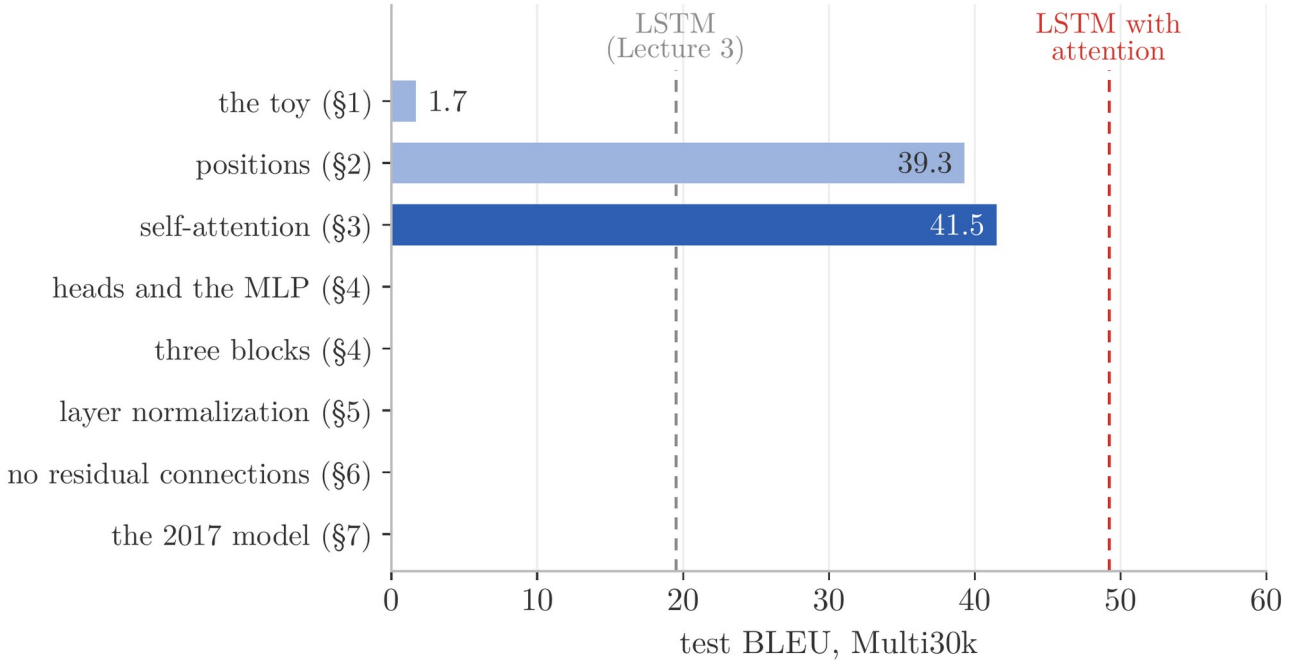
Context: Self-Attention

What self-attention changes: 39.3 → 41.5 BLEU

the experiment: one self-attention layer a side, trained as before; cross-entropy 1.42 → 1.40, BLEU **39.3 → 41.5**

uniform becomes tenue: read with karate; the end of the caption is still wrong

a smaller gain than positions: one read per layer, of one kind, which can only average (§4)



Heads, the MLP, and a Stack of Blocks

Several heads

the problem: one read per layer gathers one kind of information → **the fix here**: several reads side by side

tenue needs uniform and karate: one set of weights blurs them

H heads: each a whole attention with its own maps to d/H coordinates; W_o mixes their reads

$$\text{MultiHead}(X) = [C^{(1)}, \ldots, C^{(H)}]\, W_O$$

the same cost as one head: $4d^2$ weights and T^2d multiplications, whatever H is

what they buy: one head of 512 scores 0.9 BLEU below eight of 64; in BERT, heads specialize in the previous token, next token, punctuation (Clark et al., 2019)

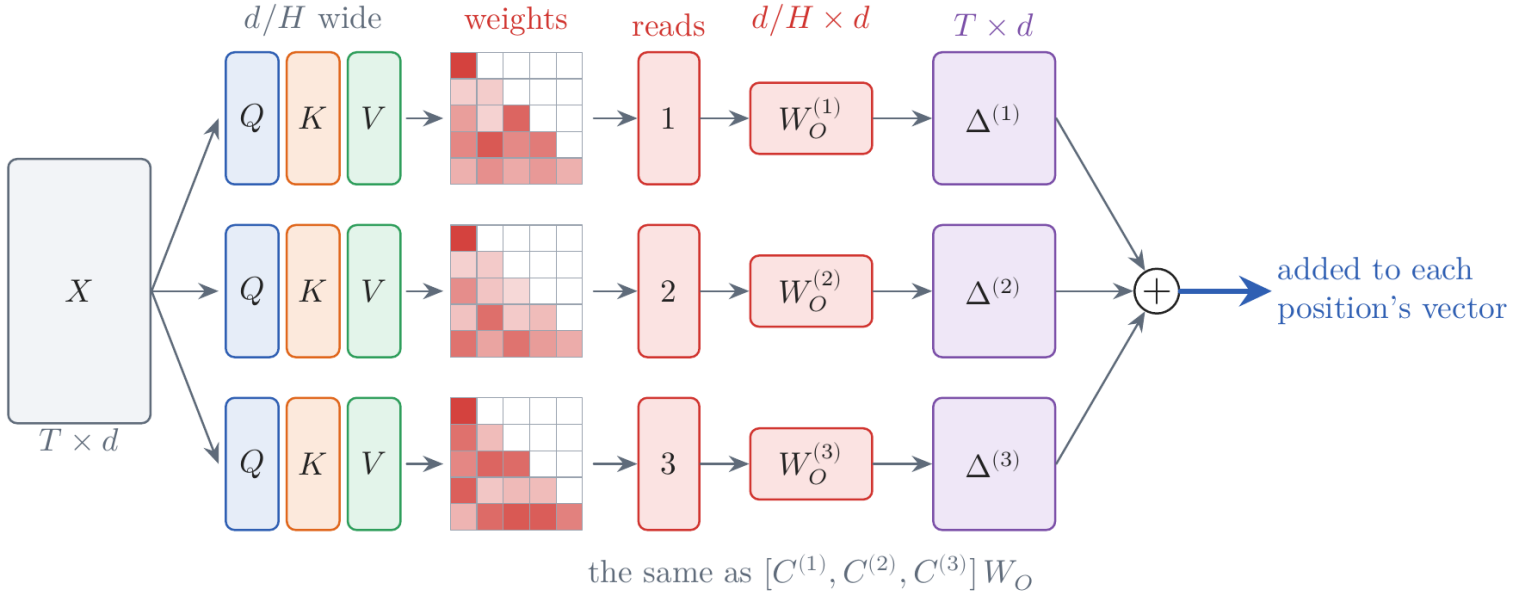


Figure 16: three heads; their changes (purple) are added

Heads, the MLP, and a Stack of Blocks

Why an MLP: communication and computation

the problem: attention only averages what it reads, linearly → **the fix here**: a network at every position

tenue needs karate AND uniform: a new feature, present only when both words are

an AND is not linear: 0 for neither word and for each alone forces 0 for both; it needs a threshold

attention is communication: it moves information between positions

the MLP is computation: one small network at every position, separately

one hidden layer, 4d wide: ReLU (2017), GELU later; two thirds of a block's weights

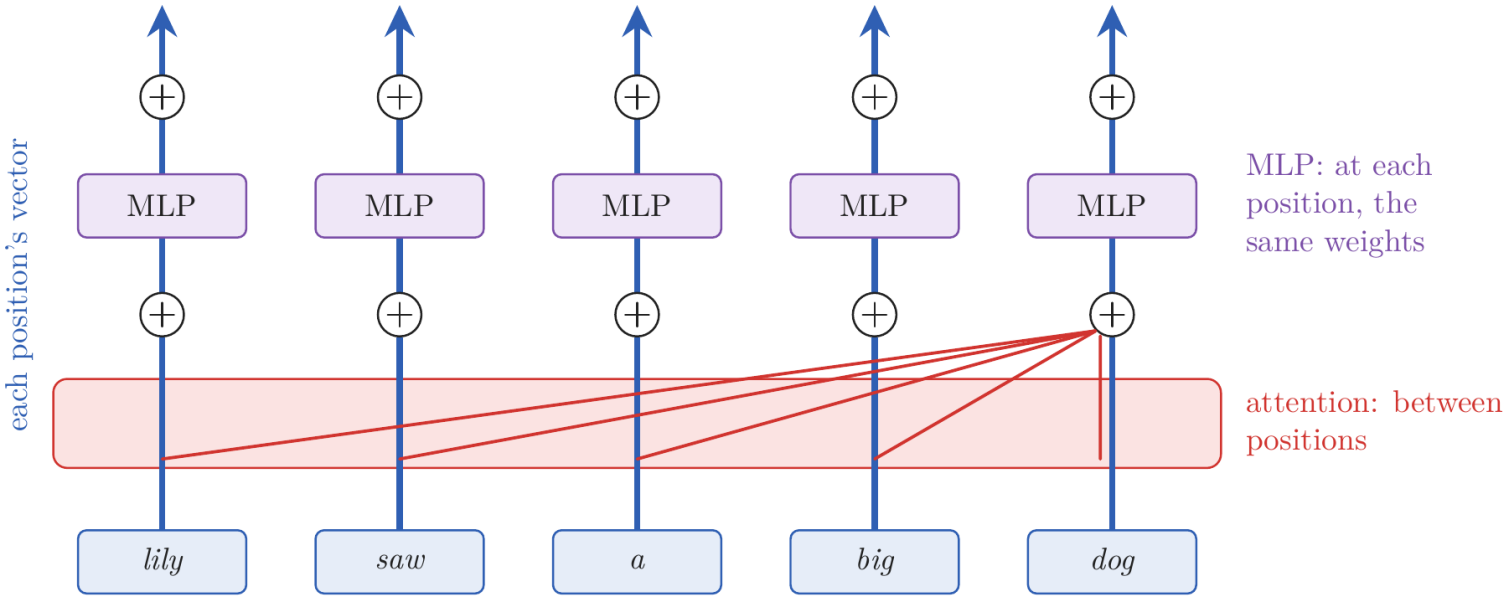


Figure 17: one block over five positions

$$\mathrm{MLP}(\mathbf{h}) = \phi(\mathbf{h}W_1 + \mathbf{b}_1)\,W_2 + \mathbf{b}_2$$

Heads, the MLP, and a Stack of Blocks

One hidden unit at a time

a sum over hidden units: column w_j of W_1 asks; row u_j of W_2 writes

$$\mathrm{MLP}(\mathbf{h}) = \sum_{j=1}^{d_{\mathrm{ff}}} \phi(\mathbf{h}\mathbf{w}_j^\top + b_{1j})\,\mathbf{u}_j + \mathbf{b}_2$$

a threshold makes an AND: only big and dog together pass the ReLU; so too karate and uniform

a key–value memory in the weights: Michael + Jordan → basketball (Sanderson, 2024c)

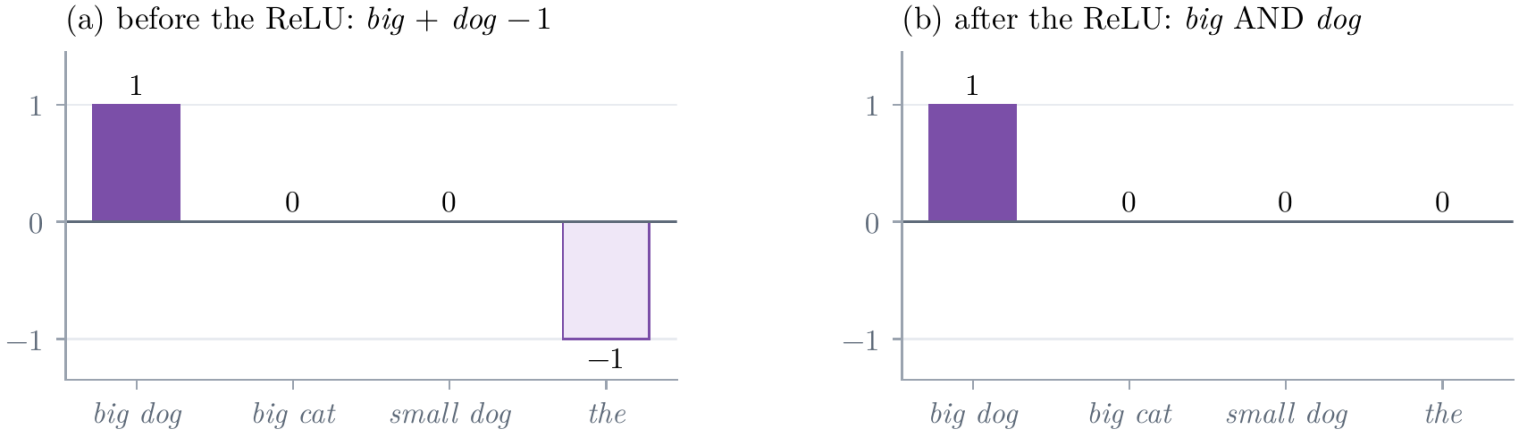


Figure 18: a toy hidden unit, bias −1

Heads, the MLP, and a Stack of Blocks

The block, and a stack of blocks

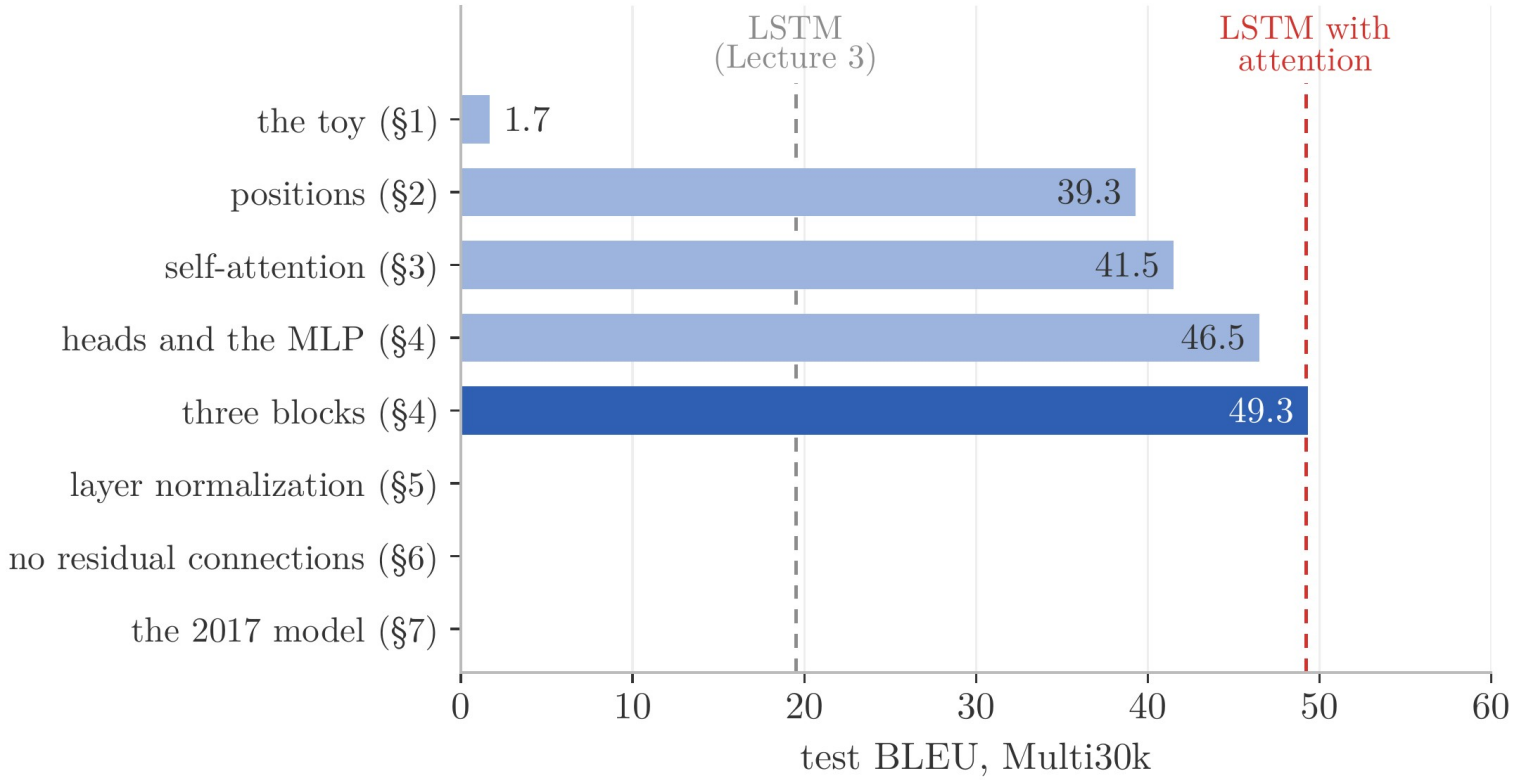
the problem: one block reads each sentence once → **the fix here**: a stack of blocks, each reading the one below

a block: attention, then the MLP, each added to the vector it read

heads and the MLP: cross-entropy 1.40 → 1.26, BLEU **41.5 → 46.5**

a stack of blocks: later reads use what earlier ones found; every decoder block reads the encoder's last, Z

three blocks a side: cross-entropy 1.16, BLEU **49.3**, past the LSTM with attention (49.2)



Heads, the MLP, and a Stack of Blocks

Nothing controls the size of the vectors

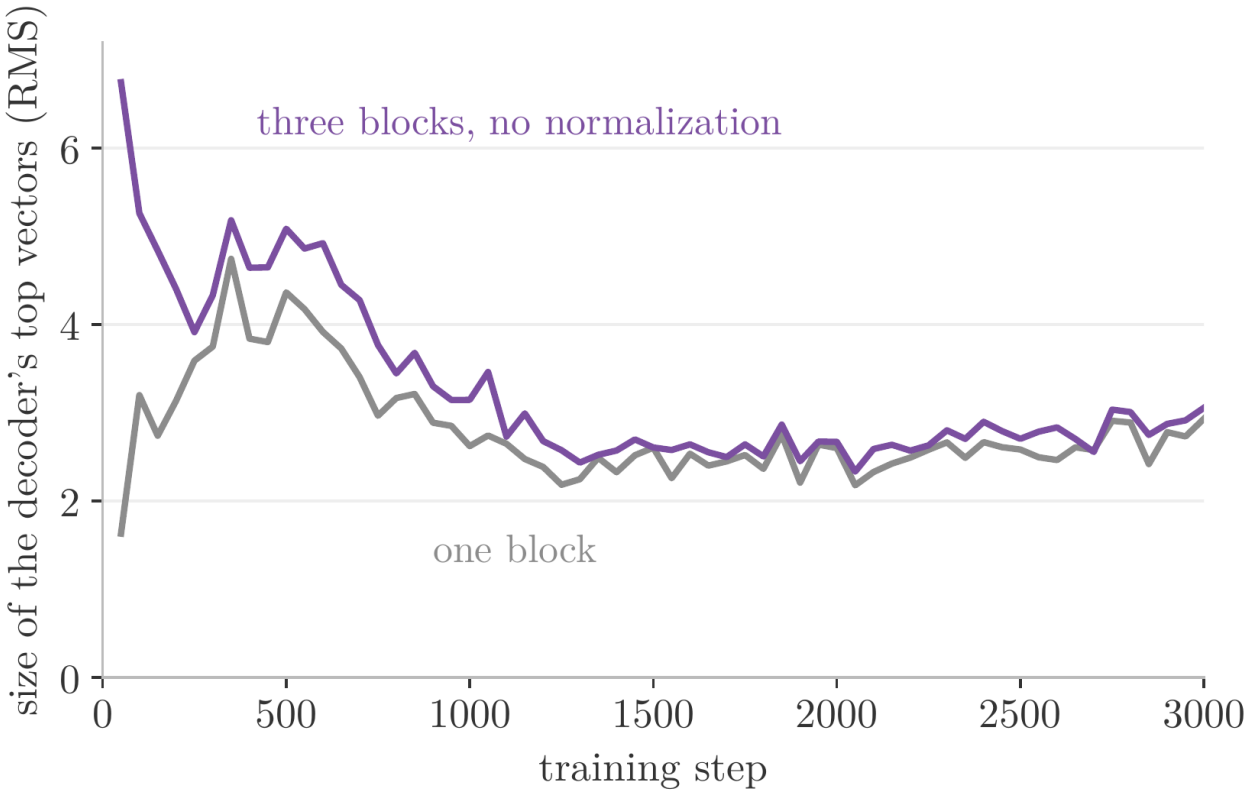
the size of a vector: the root mean square of its d numbers; about 1 if they are about ±1, whatever d

$$\text{size}(\mathbf{h}) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} h_i^2}$$

the measurement: the size of the decoder's top vectors, every 50 steps

three blocks: 6.7 after 50 steps, 3.1 at the end; one block, 1.6 and 2.9

it drifts as the weights change: every block reads vectors whose size the blocks below set



Keeping the Vectors in Check: Layer Normalization

Layer normalization: one vector at a time

the problem: the size of the vectors drifts, and much depends on it → **the fix here**: rescale each vector by itself

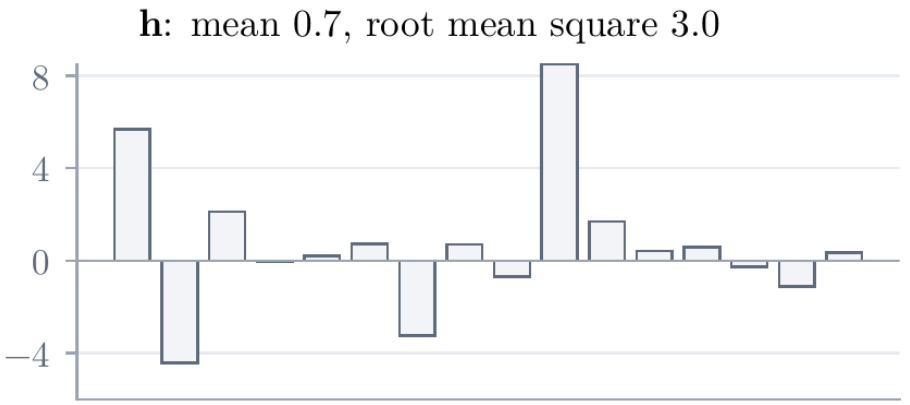
the size matters: in the scores (§3), the MLP's thresholds, and the step sizes

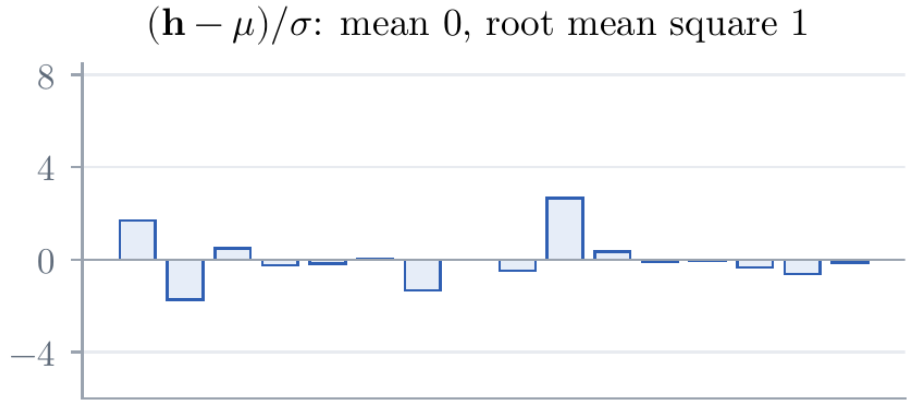
layer normalization: μ and σ² are the mean and variance of the vector's own d numbers

$$\text{LayerNorm}(\mathbf{h}) = \gamma \odot \frac{\mathbf{h} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

mean 0, size 1, whatever h was: the pattern is kept; γ and β, learned, rescale it

one vector at a time: the same in training and in use; any length; batches of one





Keeping the Vectors in Check: Layer Normalization

Normalizing after each addition

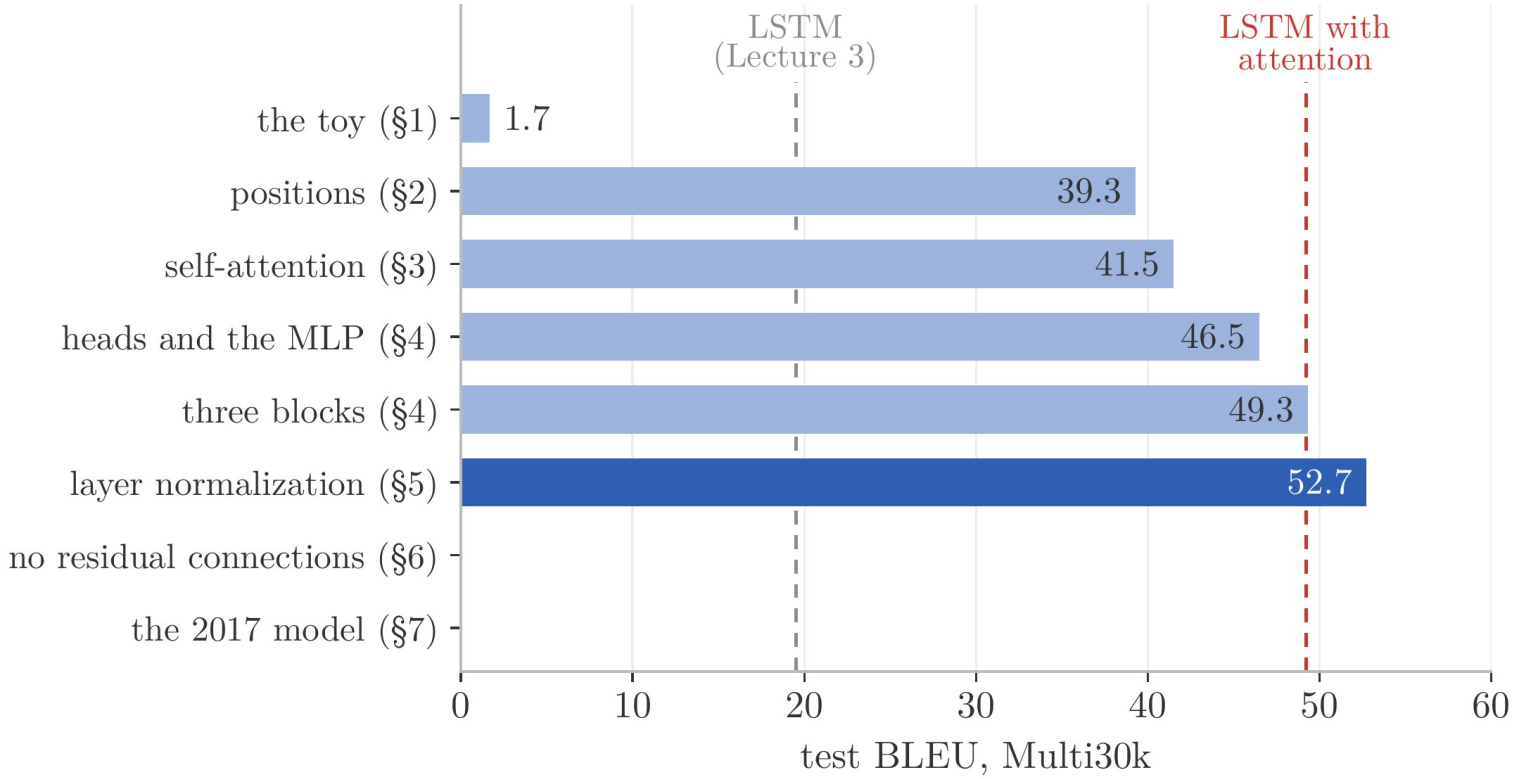
after each addition: every sublayer reads a vector whose size γ and β set

$$\mathbf{h} \leftarrow \mathrm{LayerNorm}(\mathbf{h} + F(\mathbf{h}))$$

the experiment: the three-block translator, normalized after each of its 15 additions; the size now **grows smoothly**

cross-entropy 1.16 → 1.08, BLEU **49.3 → 52.7**

but why does the stack train at all? That depends on the additions of §3



Depth: Residual Connections

Why the stack trains: the additions of §3

the question: why does the stack of §4 train at all? → **the answer**: the additions of §3, from residual networks

without the additions, three normalized blocks a side give **0.0 BLEU** (*un un un* …)

deeper is worse (He et al., 2016a): deeper plain networks fit their training data worse

our repeat on the digits: 44 plain layers still misclassify **60%** of the training images

their remedy, our §3: learn a change, not a replacement

$$\mathbf{h} \leftarrow \mathbf{h} + F(\mathbf{h})$$

every depth fits: 44 layers reach a test error of 3.2%

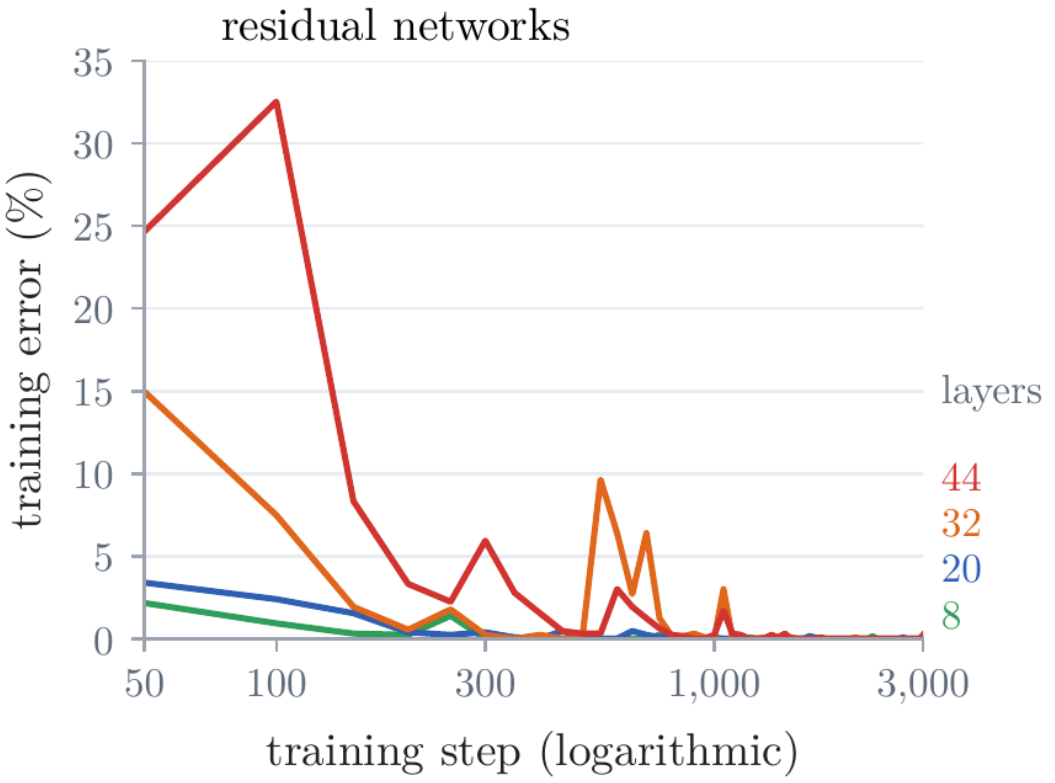


Figure 22b: residual networks of the same depths; the error axis stops at 35%

Depth: Residual Connections

The gradient's path through the additions

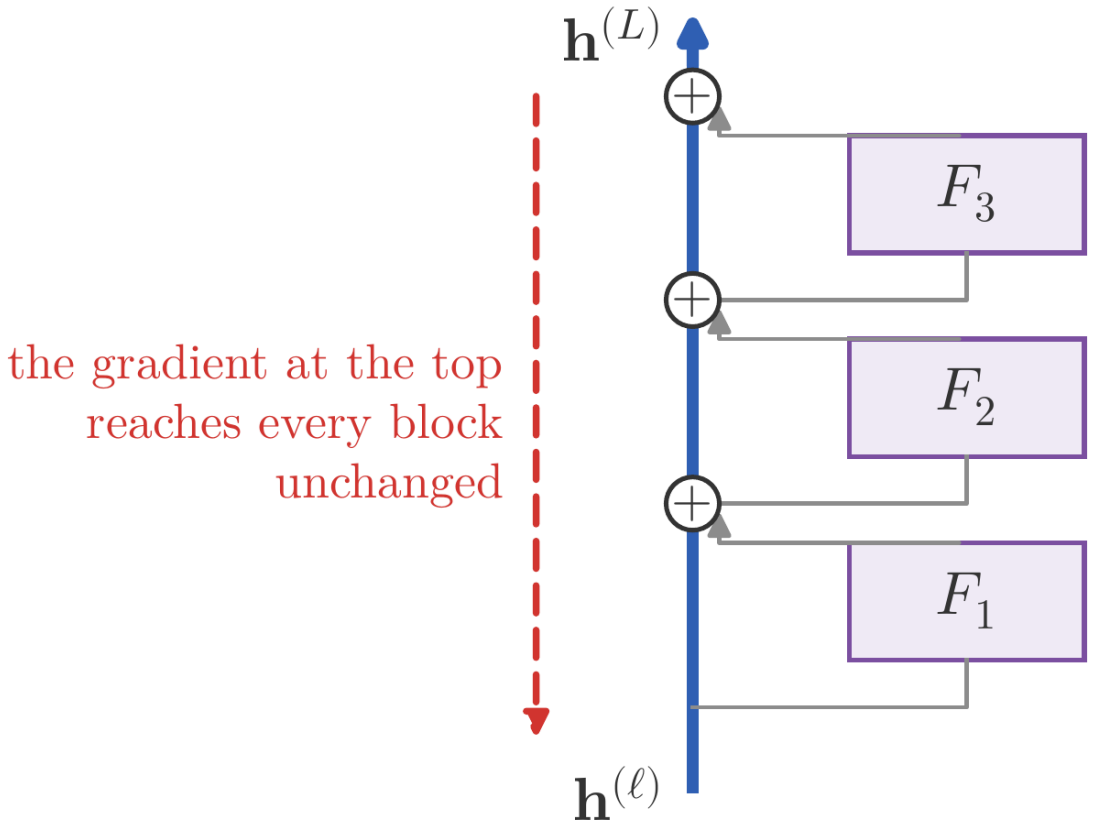
the top is the bottom plus what was added

$$\mathbf{h}^{(L)} = \mathbf{h}^{(\ell)} + \sum_{i=\ell}^{L-1} F_i(\mathbf{h}^{(i)})$$

so the gradient has an identity term: it reaches every block unchanged

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}} \left(I + \frac{\partial}{\partial \mathbf{h}^{(\ell)}} \sum_{i=\ell}^{L-1} F_i(\mathbf{h}^{(i)}) \right)$$

the LSTM's memory cell, turned to depth: an additive path, along the layers instead of the positions (Lecture 2)



Depth: Residual Connections

Averages of averages become alike

attention alone collapses: stacked averages converge to identical rows (Dong et al., 2021)

our translator, no additions: every head gives every word the same weight, **1/13**

a character model: 8 blocks, children's stories; without additions, two positions' vectors reach cosine **0.99** at initialization

it learns only the frequencies: 4.22 bits per character; with additions **1.76**

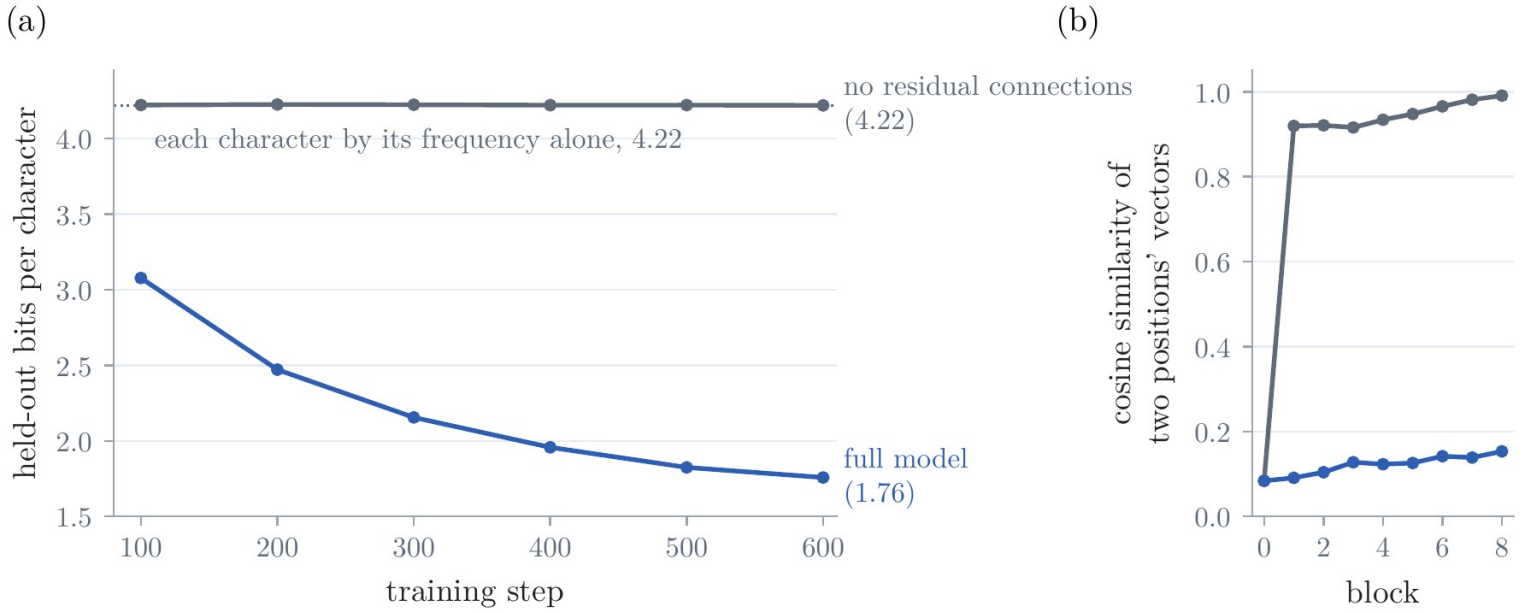


Figure 23: (a) held-out bits per character; (b) cosine similarity of two positions' vectors, by block

Depth: Residual Connections

Pre-norm and post-norm

post-norm (2017): a normalization on the residual path

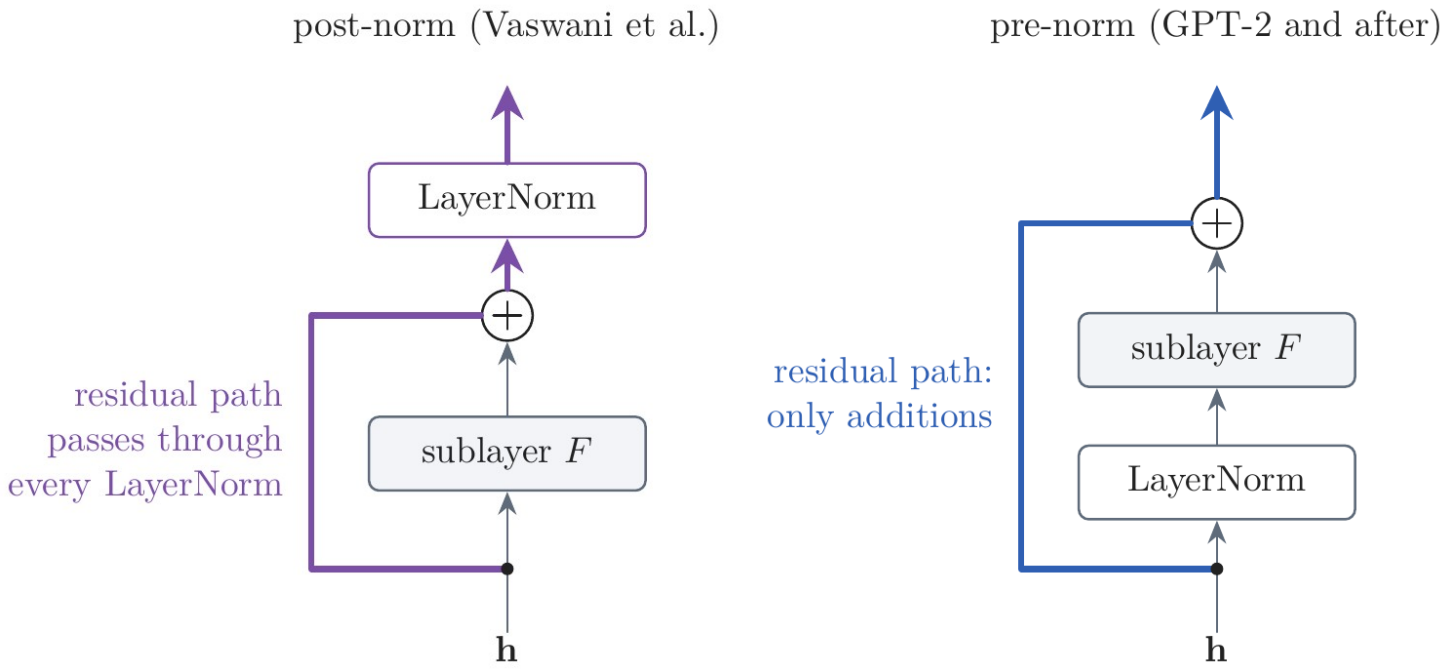
$$\mathbf{h} \leftarrow \text{LayerNorm}(\mathbf{h} + F(\mathbf{h}))$$

pre-norm (GPT-2 on): only additions on the residual path

$$\mathbf{h} \leftarrow \mathbf{h} + F(\text{LayerNorm}(\mathbf{h}))$$

post-norm needs a warmup: without one, 8.5 BLEU against about 34 (Xiong et al., 2020); our character model never gets past the frequencies

trained, post-norm can end ahead: 27.6 against 26.8 BLEU (Nguyen and Salazar, 2019)



Depth: Residual Connections

The residual stream

a shared workspace: d numbers at each position; every sublayer reads it and adds to it (Elhage et al., 2021)

in pre-norm, the top is the sum: the token's vector, plus one change per head and per MLP

L blocks, at most L moves: only attention moves information between positions

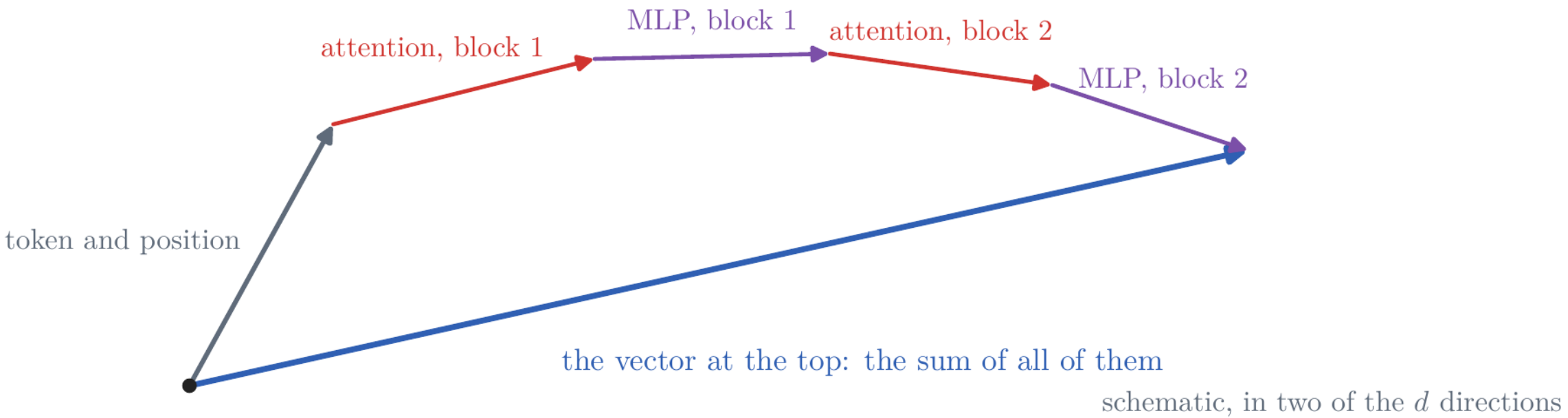


Figure 26: gray the token's vector, red each attention's change, purple each MLP's, blue the sum

The Transformer of 2017

The translator so far: every part in place

positions (§2): each word's vector plus its position's

self-attention (§3), four heads (§4): each read added to the vector it came from

an MLP, three blocks a side (§4): every decoder block reads Z, the encoder's output

add, then normalize (§§5–6): after every sublayer; the stack trains

Vaswani et al.'s encoder–decoder, at d = 192: **52.7 BLEU**; §7 adds a shared table

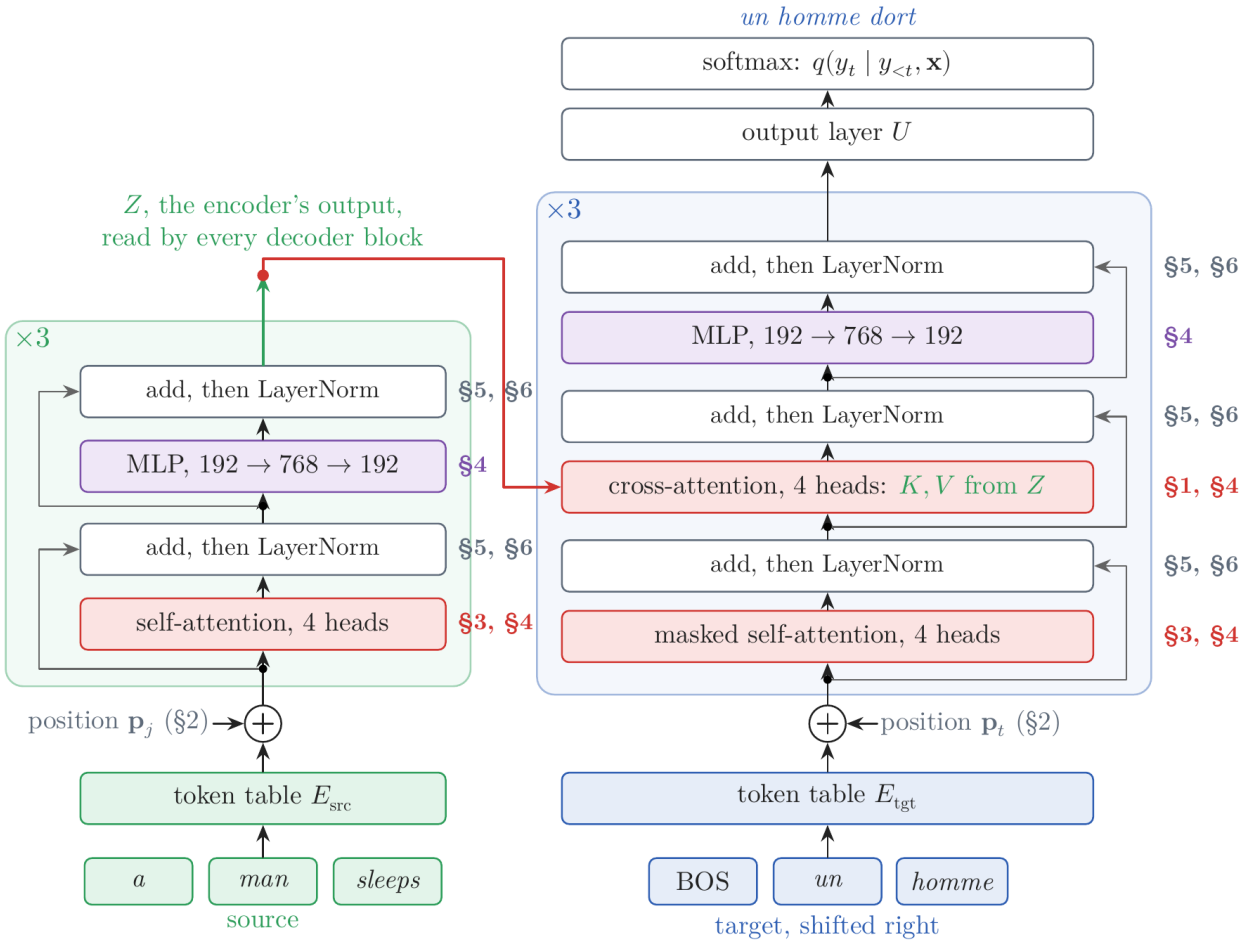


Figure 27: the translator at the end of §6; gray arrows, the additions

The Transformer of 2017

Three attentions

encoder self-attention: every source position reads every source position

masked decoder self-attention: every target position reads those up to its own

cross-attention: queries from the decoder; keys and values from Z, the encoder's last layer

$$Z \leftarrow \text{LayerNorm}(Z + \text{MultiHead}(Z, Z))$$
$$Z \leftarrow \text{LayerNorm}(Z + \text{MLP}(Z))$$

$$Y \leftarrow \text{LayerNorm}(Y + \text{MultiHead}_{\text{causal}}(Y, Y))$$
$$Y \leftarrow \text{LayerNorm}(Y + \text{MultiHead}(Y, Z))$$
$$Y \leftarrow \text{LayerNorm}(Y + \text{MLP}(Y))$$

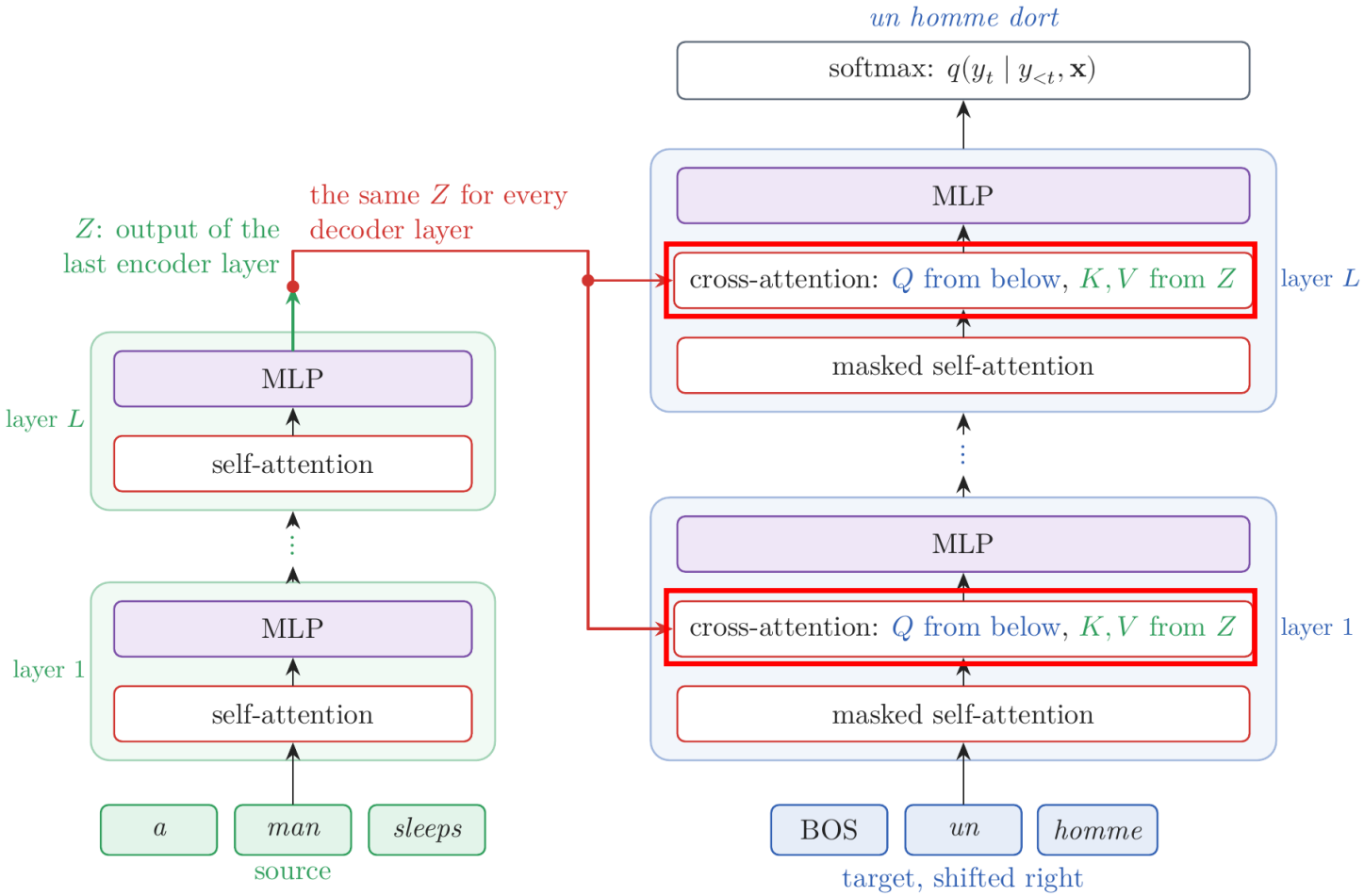


Figure 19: normalizations and residual connections not drawn

The Transformer of 2017

One table for the input and the output

the problem: the stack starts to fit its training captions too closely; a third of its weights are two French tables → **the fix here**: share one table

the output layer is a table of word vectors:
the logit of word v is h_t · column v

tie them: better and smaller, perplexity 74.3 tied (51M), 78.4 untied (66M) (Press and Wolf, 2017)

$$q(y_t \mid y_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{h}_t E^\top)$$

here: the French table is also the output layer, 1,241,762 fewer weights

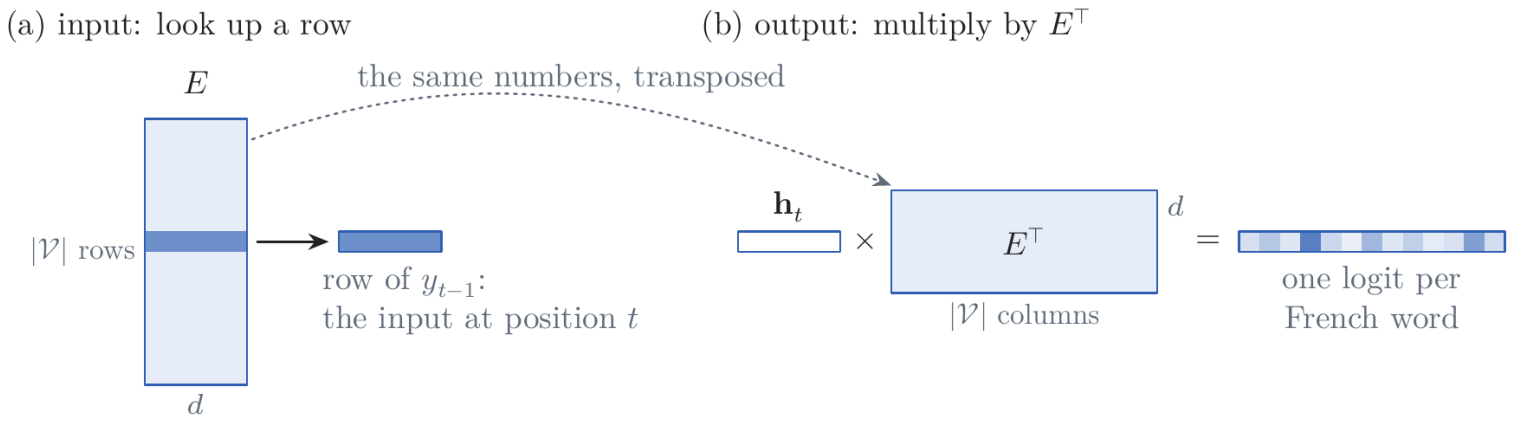


Figure 28: one table, at the input and, transposed, at the output

The Transformer of 2017

The paper's result

two sizes: base, 6 layers a side, d = 512, 8 heads, 65 million parameters; big, d = 1,024, 16 heads, 213 million

trained on WMT 2014: English–German, 4.5 million pairs; one machine with eight GPUs, 12 hours (base), 3.5 days (big)

better and cheaper: base **27.3** BLEU, above every earlier system, at a third of the cheapest's cost; big **28.4**

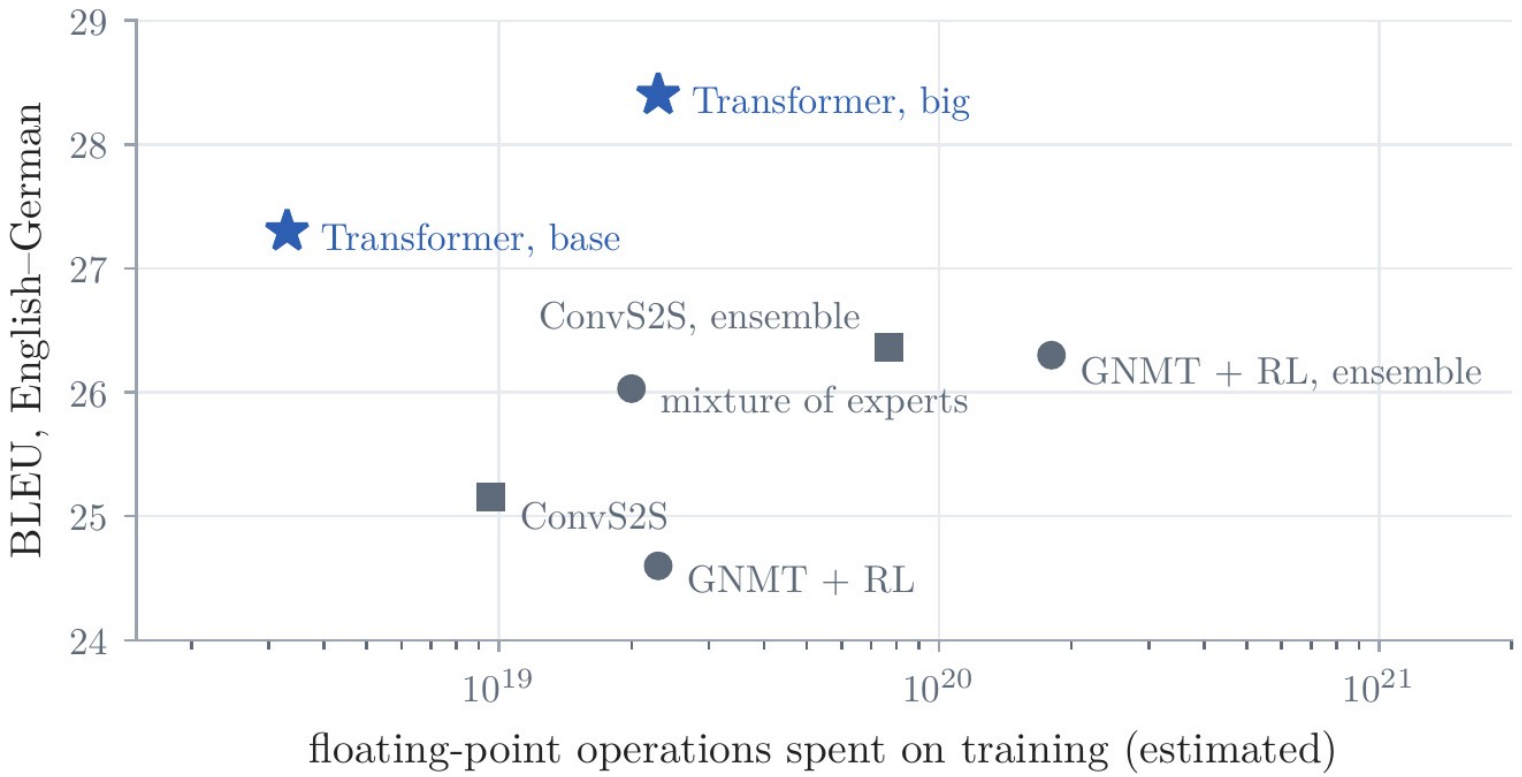
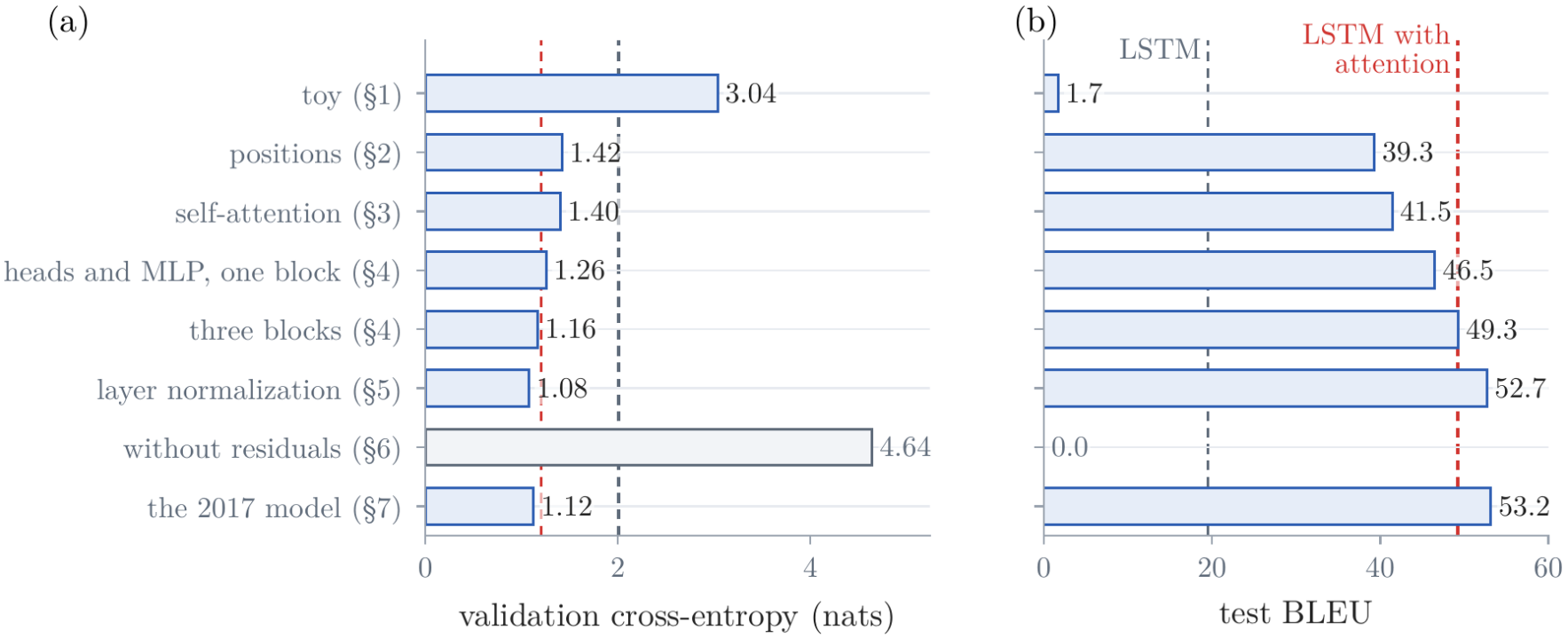


Figure 29: from Vaswani et al. (2017), Table 2; higher and further left is better

The Transformer of 2017

What each part contributed

the 2017 model: BLEU 52.7 → **53.2**, ahead of the LSTM with attention (49.2); cross-entropy 1.08 → 1.12



positions with the learned maps mattered most; each later part lowered the cross-entropy

single runs, in one order: what each step of this build did, not what each part is worth alone

removing one part from a complete model: residual connections cost the most, the MLPs the least

The Transformer of 2017

The karate caption at every step

English	*a girl in karate uniform breaking a stick with a front kick .*	test BLEU
the toy (§1)	*un bâton dans un bâton dans un bâton dans …*	1.7
positions (§2)	*une fille en uniforme de karaté avec un bâton .*	39.3
self-attention (§3)	*une fille en tenue de karaté avec un ballon de pied devant un coup .*	41.5
heads and the MLP (§4)	*une fille en uniforme de karaté fait un bâton devant un coup .*	46.5
three blocks (§4)	*une fille en tenue de karaté faisant un grand saut avec un coup .*	49.3
layer normalization (§5)	*une fille en uniforme de karaté milieu un bâton avec un ballon .*	52.7
the 2017 model (§7)	*une fille en tenue de karaté fait un bâton avec un ballon .*	53.2
reference	*une fille en tenue de karaté brisant un bâton avec un coup de pied .*	

Figure 31: greedy output of each translator of §§1–7; the reference in gray

one caption improves unevenly: positions end the loop, self-attention finds *tenue*, later steps trade one error for another

the average rises at every step: BLEU over the 1,000 test captions

Break

Lecture 4: Transformers

1. **Building the Transformer**

   from Lecture 3's toy translator to the encoder–decoder of 2017, one part at a time

2. **The language model**

   the decoder alone, trained on children's stories; positions as rotations

3. **Strengths, limitations, and variants**

   every weight and its cost; the key–value cache; what it does well and where it falls short; variants

The Decoder Alone: a Language Model

Decoders, encoders, and their uses

the problem: most text comes without a translation → **the fix here**: drop the encoder, and predict the next token of plain text

the decoder alone is a language model:
Lecture 2's model, trained on plain text

$$q(x_t \mid x_{<t}) \quad \text{at every position } t$$

GPT (2018), GPT-2 (2019): 12 blocks and 512 tokens; then pre-norm, 48 blocks, 1.5 billion parameters

encoders fill in blanks: BERT reads both ways and predicts 15% hidden tokens, which do not make a distribution over texts

since about 2020: nearly every large language model is a decoder

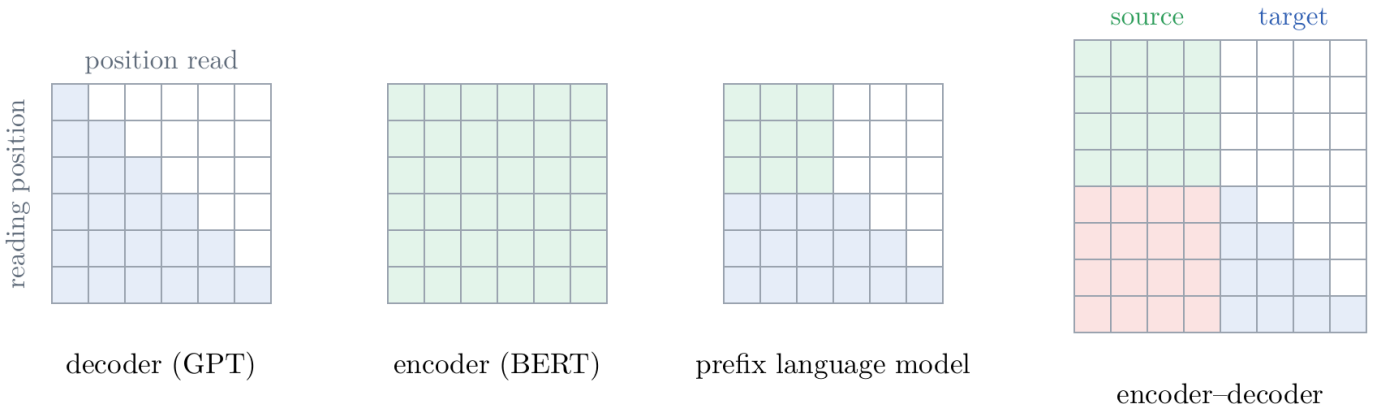


Figure 33: which positions (columns) each position (row) may read

The Decoder Alone: a Language Model

The word model

a decoder-only Transformer: GPT-2's layout, pre-norm; trained on Lecture 2's children's stories

small: L = 4 blocks of d = 128; 4 heads of 32; MLPs of 512; 8,682 tokens; 1.9 million parameters

positions: none added to the vectors; Q and K are rotated by their positions (§9)

a batch: 16 chunks of 256 tokens; position t holds x_{t-1} and predicts x_t

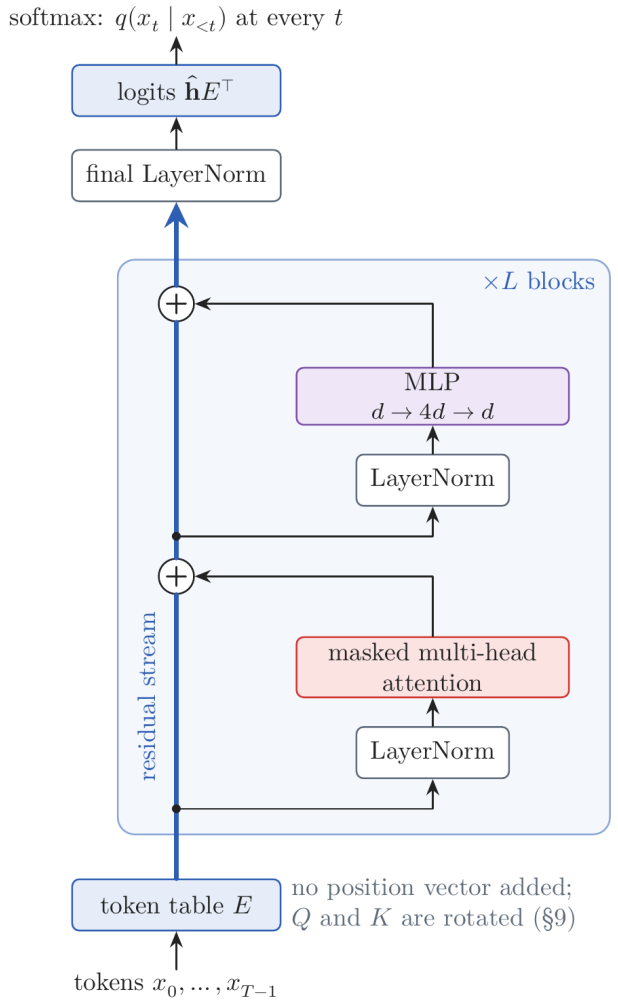


Figure 34: the decoder, pre-norm

The Decoder Alone: a Language Model

One batch, step by step

step	operation	shape
embed		
tokens	I: token ids, the inputs $x_0, \ldots, x_{T-1}$ of each chunk	16×256
embed	$X = E[I]$, rows of the token table	$16 \times 256 \times 128$
attention sublayer, in each of the L blocks		
normalize	$\hat{X} = \mathrm{LayerNorm}(X)$, each vector separately	$16 \times 256 \times 128$
project	$Q, K, V = \hat{X}W_{Q,K,V} + \mathbf{b}$, 4 heads; rotate Q and K	$16 \times 4 \times 256 \times 32$
score	$S = QK^\top/\sqrt{32} + M$	$16 \times 4 \times 256 \times 256$
weigh	$A = \mathrm{softmax}(S)$, along each row	$16 \times 4 \times 256 \times 256$
read	$C = AV$, then the heads side by side	$16 \times 256 \times 128$
write	$X \leftarrow X + CW_O + \mathbf{b}_O$	$16 \times 256 \times 128$
MLP sublayer, in each of the L blocks		
expand	$G = \mathrm{GELU}(\mathrm{LayerNorm}(X)W_1 + \mathbf{b}_1)$	$16 \times 256 \times 512$
write	$X \leftarrow X + GW_2 + \mathbf{b}_2$	$16 \times 256 \times 128$
output, after the blocks		
logits	$Z = \mathrm{LayerNorm}(X)\,E^\top$, the token table in reverse	$16 \times 256 \times 8{,}682$
loss	average of $-\log \mathrm{softmax}(Z)$ at the targets $x_1, \ldots, x_T$	scalar

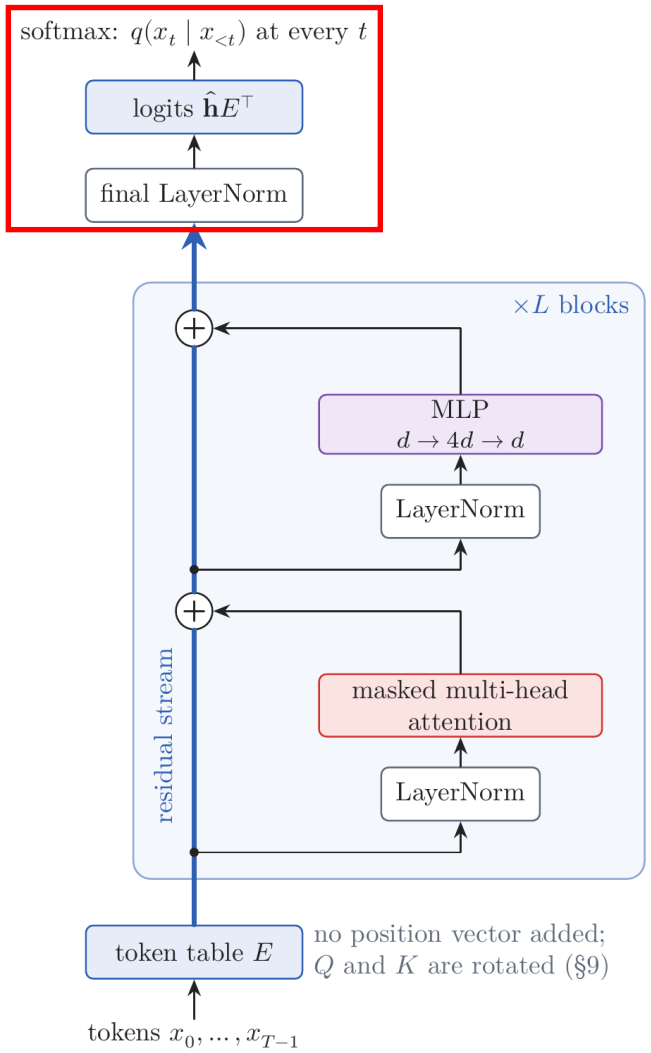


Figure 34: the decoder

The Decoder Alone: a Language Model

Training: every prediction in one pass

one pass, every prediction: each from its own prefix (§3)

Training the word model

1. **for** step $= 1, \ldots, 4{,}000$ **do**
2. sample $B = 16$ chunks of $T = 256$ tokens
3. forward: the logits at all BT positions
4. loss $\leftarrow$ mean of $-\log q(x_t \mid x_{<t})$
5. backward: the gradient (backpropagation)
6. clip it at norm 1; an Adam step
7. **end for**
8. keep the weights best on held-out stories

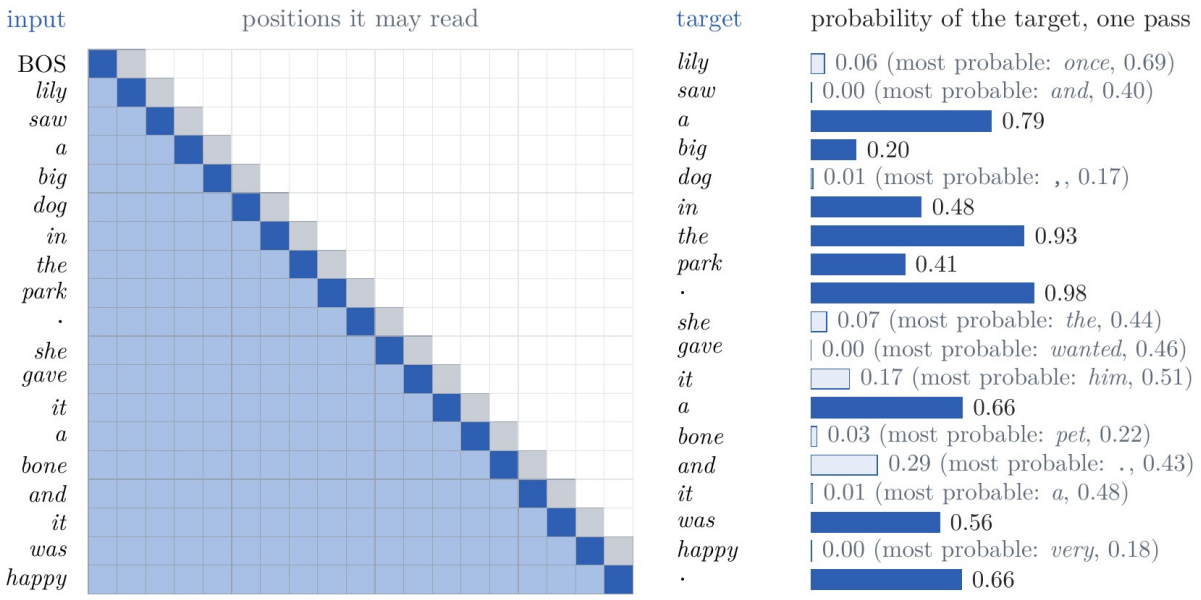


Figure 35: inputs (rows), the causal mask, each target and its probability

the word model: 4,000 steps, the 16 million words Lecture 2's LSTM read; 54 minutes on two processor cores

against Lecture 2: test perplexity **9.9**, the LSTM 23.1; two setups, not attention against recurrence

The Decoder Alone: a Language Model

Generation

Generation, sampling at temperature τ

$x \leftarrow$ (BOS)
repeat
    $\mathbf{z} \leftarrow$ the model's logits at the last position of x
    sample a token from softmax($\mathbf{z}/\tau$); append it to x
until the end-of-story token, or n tokens
return x

a sample from BOS, temperature 0.7

"once upon a time, there was a little girl named lily. she loved to play outside in the sun. one day, she went to the park with her mommy and daddy. they were having so much fun! ..."

sharp where the story allows few words: the five most probable next words after three prompts

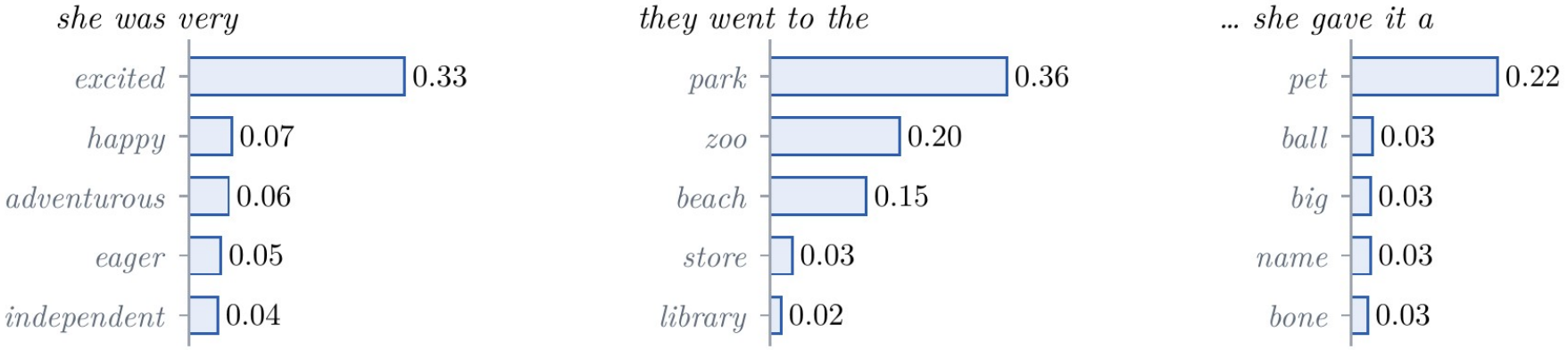


Figure 36: the five most probable next words, with their probabilities

The Decoder Alone: a Language Model

Checking the mask

a leak copies the answer: teacher-forced perplexity **1.08** against the honest model's 10.0

test 1, change the future: no earlier output may change; leaky, up to 9.2 nats, four positions back

test 2, one pass against one prefix at a time: honest, within 3×10^{-7}; leaky, 0.09 against 5.85

test 2 is the definition: in use, a model sees only prefixes

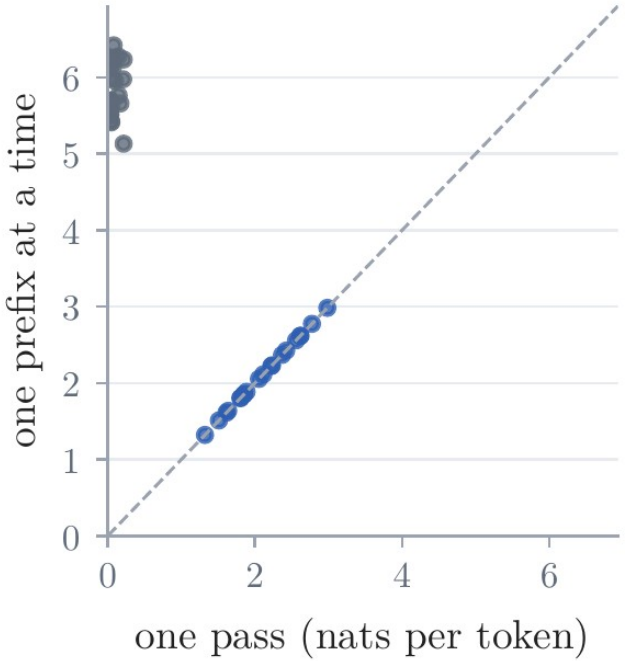


Figure 37b: log loss, one pass against one prefix at a time

Positions Revisited: Rotations

What a relative position must be

the problem: added position vectors mix where a word stands with what it is; they did not extend beyond the training length → **the fix here**: put the positions into the scores

the score mixes words and positions

$$e_{tj} = \left(E(x_{t-1}) + \mathbf{p}_t\right) W_Q W_K^\top \left(E(x_{j-1}) + \mathbf{p}_j\right)^\top$$

ask for scores that depend only on the offset: linear, offsets only, nothing at zero, continuous (Su et al., 2021; Puranik, 2026)

a one-parameter group

$$F_t G_j^\top = B(j-t) \quad \Rightarrow \quad B(a+b) = B(a)\,B(b),$$
$$B(u) = \exp(uX)$$

so rotate, decay, or both: only the frequencies remain to choose

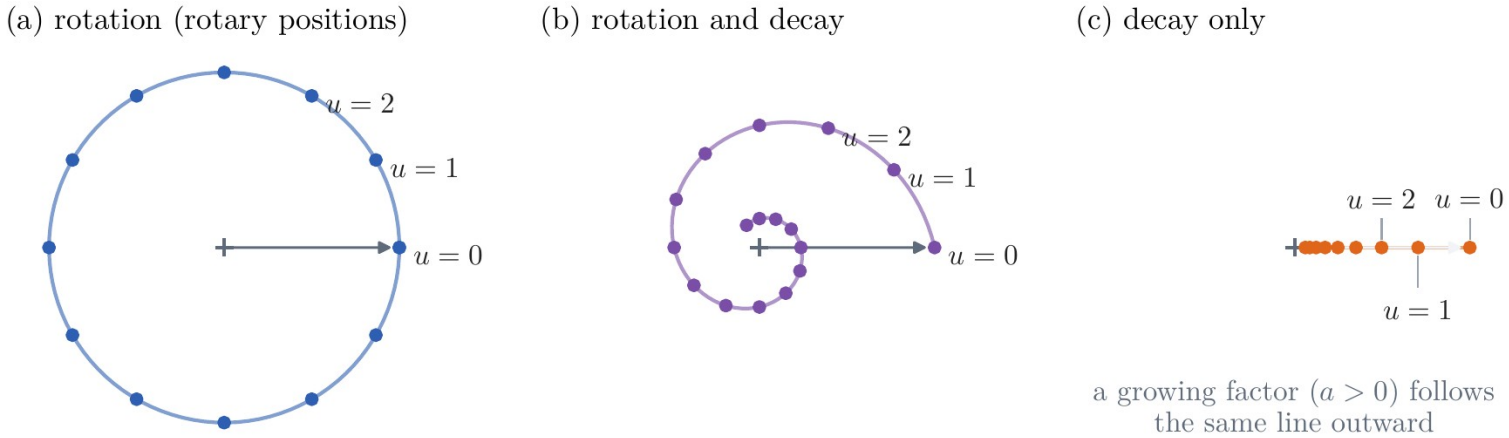


Figure 38: what a one-parameter group does to a pair of coordinates, by offset u

Positions Revisited: Rotations

Rotary position embeddings

rotate the query and the key: pair i of position t turned by t ω_i; nothing added to the word vectors

the score sees only the offset

$$(\mathbf{q}_t R_t)(\mathbf{k}_j R_j)^\top = \mathbf{q}_t\, R_t R_j^\top\, \mathbf{k}_j^\top = \mathbf{q}_t\, R_{t-j}\, \mathbf{k}_j^\top$$

"two positions back" is the same everywhere: the values carry no positions

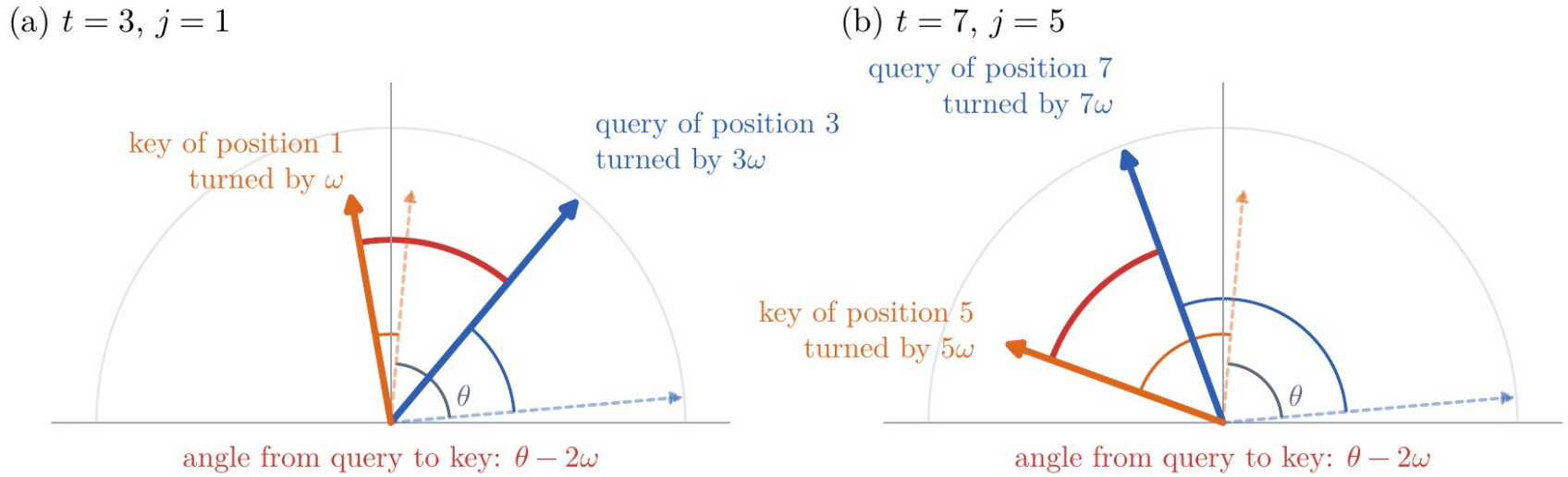


Figure 39: query at 3 and key at 1, then at 7 and 5; the same angle

Positions Revisited: Rotations

Without positions, and beyond the training length

the experiment: a four-block character model, 750 steps, with no positions, learned vectors, or rotary positions

rotary did best: 1.57 bits per character; learned 1.93; none 2.19

no positions still beats the bigram (3.26): a uniform head reads 1/t of BOS, so the mask gives the position away

beyond the training length: unseen angles; rescale the rotations and train briefly on longer texts

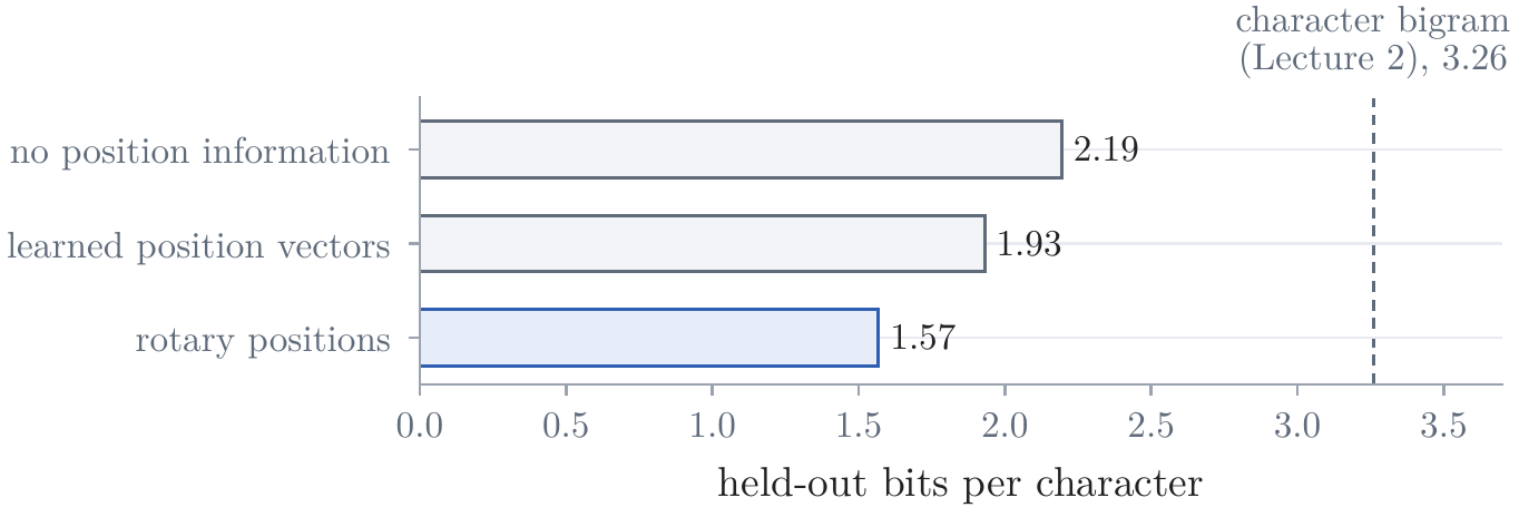


Figure 40: held-out bits per character; dashed, Lecture 2's character bigram model

Lecture 4: Transformers

1. **Building the Transformer**

   from Lecture 3's toy translator to the encoder–decoder of 2017, one part at a time

2. **The language model**

   the decoder alone, trained on children's stories; positions as rotations

3. **Strengths, limitations, and variants**

   every weight and its cost; the key–value cache; what it does well and where it falls short; variants

Every Weight, and What It Costs

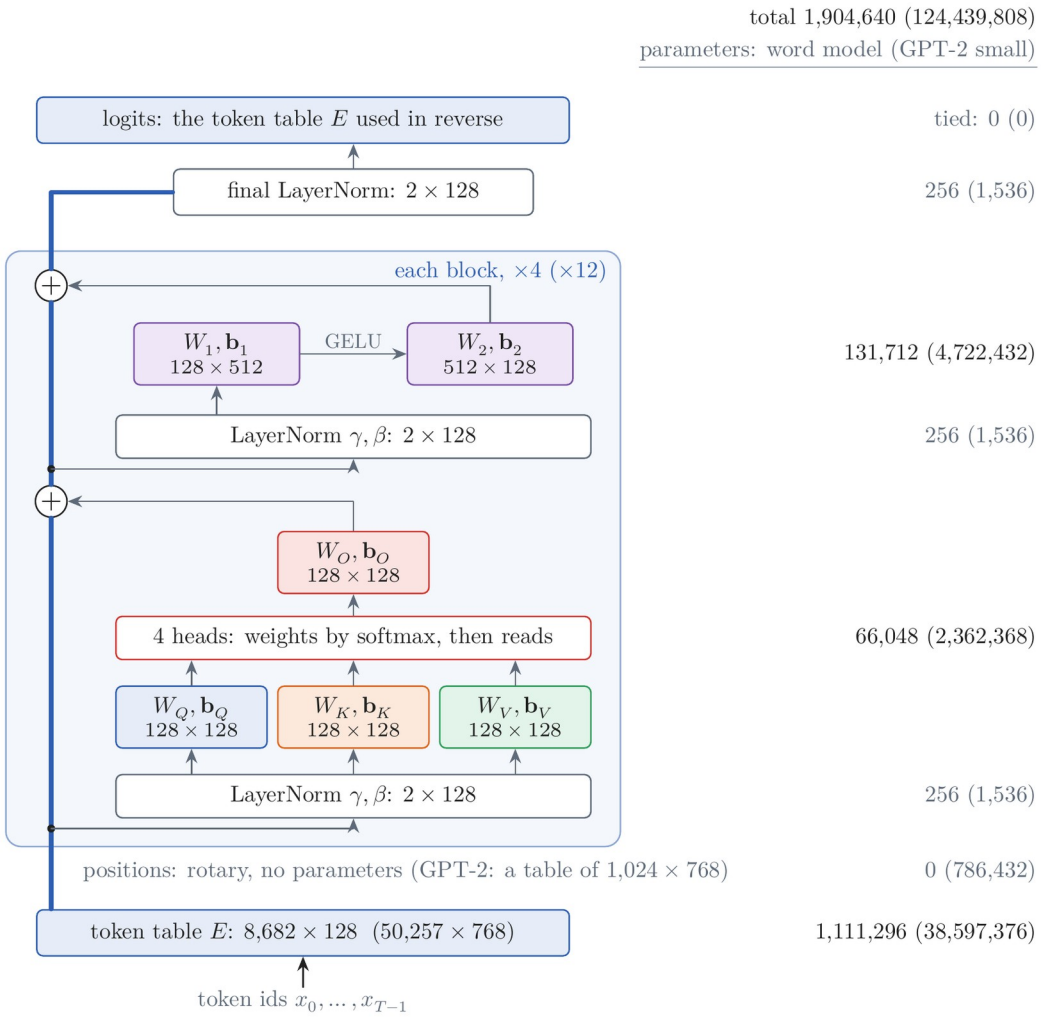
Every weight matrix

the token table: |V| × d, a vector per token; in reverse, the logits

attention, in each block: W_Q and W_K choose where each head reads, W_V and W_O what it writes

the MLP, in each block: W_1 holds its units' questions, W_2 what each adds

the normalizations: a scale and a shift; nothing else has parameters



Every Weight, and What It Costs

Parameters: $N \approx 12Ld^2$

per block: $4d^2$ in attention, $8d^2$ in the MLP

$$N \approx 12\,L\,d^2$$

GPT-2 small

GPT-2 small	count	parameters
blocks	$12 \times 12 \times 768^2$	84,934,656
biases, norms	$12 \times 13 \times 768$	119,808
token table	$50{,}257 \times 768$	38,597,376
position table	$1{,}024 \times 768$	786,432
final norm	2×768	1,536
total		124,439,808

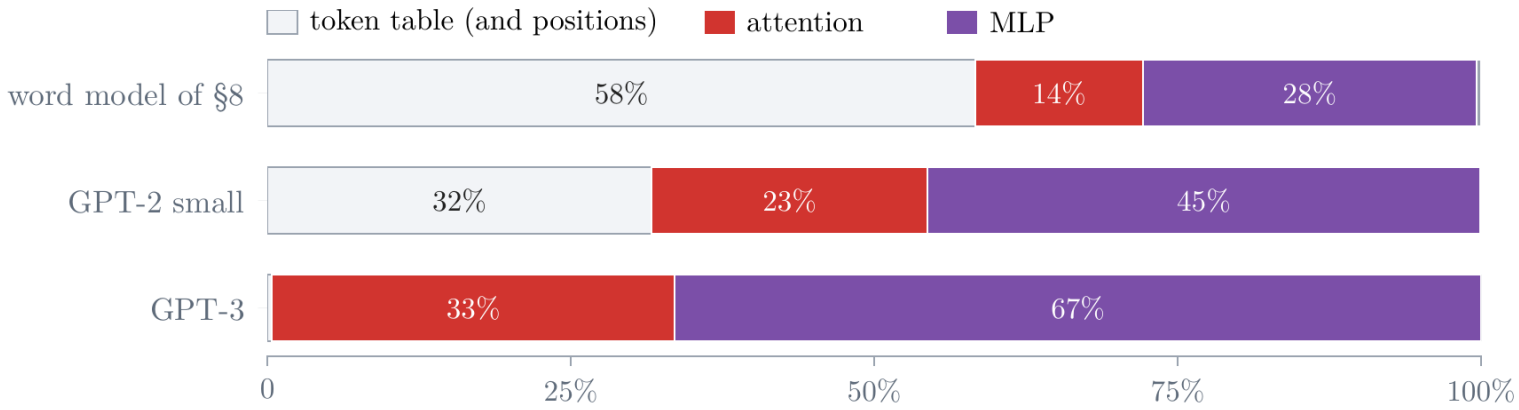


Figure 42: parameter shares of the token table, attention, and the MLPs

as models grow, the blocks take over: GPT-3 has 1.74×10^{11} in its blocks, 0.6 billion in its token table

Every Weight, and What It Costs

Computation: C ≈ 6N

forward: a multiplication and an addition per weight, so 2N FLOPs per token

backward, twice as much: the input's gradient and the weights'

$$C \approx 6N \text{ FLOPs per token}$$

a check: the base Transformer took 9.5×10^{17} FLOPs over 100,000 steps; the paper estimated 3.3×10^{18}, so its GPUs ran at about 30%

a layer $Y = XW$, with $G = \partial\mathcal{L}/\partial Y$ from the layer above

forward: the output	backward: the input's gradient	backward: the weights' gradient
$Y = XW$	$GW^\top$	$X^\top G$
2 per weight	2 per weight	2 per weight

$2 + 2 + 2 = 6$ FLOPs per weight and token: $C \approx 6N$

Every Weight, and What It Costs

The quadratic term

attention's own work: 4Td FLOPs per token and block, with T positions in view

it grows with the context

$$\underbrace{4LT^2d}_{\text{attention}} \quad \text{against} \quad \underbrace{24LTd^2}_{\text{weights}}$$

$$\text{equal at } T = 6d$$

GPT-2 small: about 4,600 tokens, beyond its 1,024

and memory: T × T weights per head, 67 million numbers at 8,192 tokens; FlashAttention never stores them

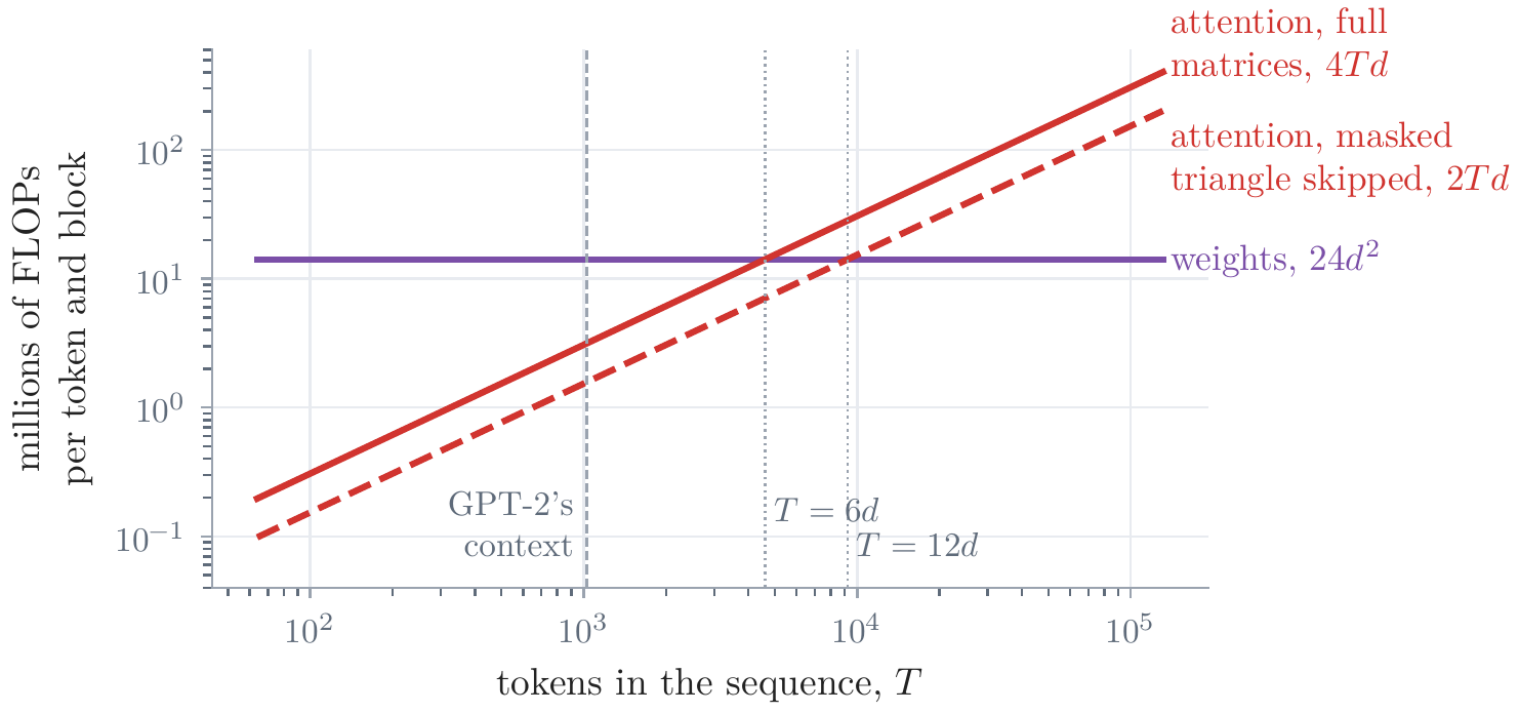


Figure 43: forward FLOPs per token in one block of GPT-2 small; purple the weights, red attention

Every Weight, and What It Costs

The key–value cache

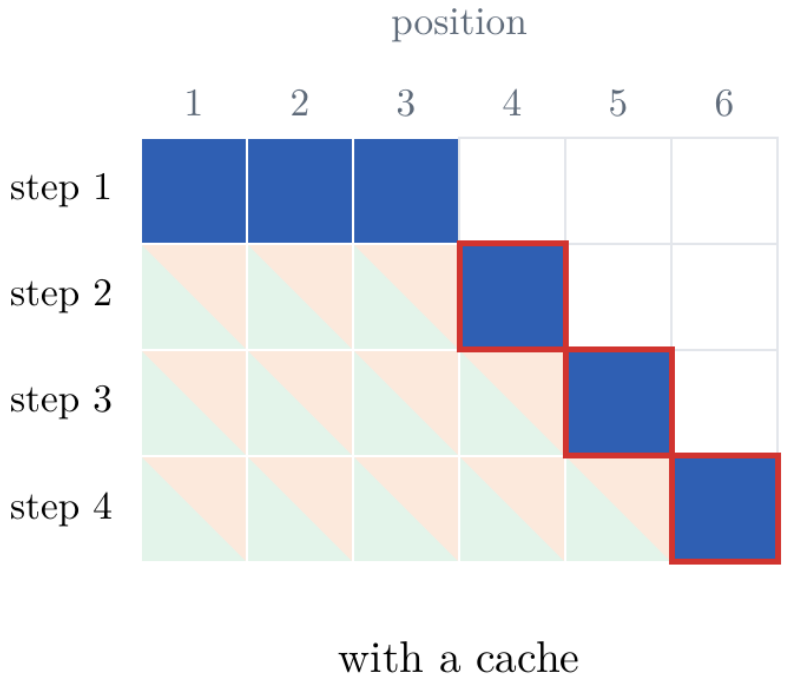
generation recomputes the prefix: at every step, to use only its last position

earlier keys and values never change: so keep them

One step of generation, with a cache

1 **for** each block $\ell = 1, \dots, L$ **do**
2 compute $\mathbf{q}, \mathbf{k}, \mathbf{v}$ of the new position only
3 append $\mathbf{k}$ and $\mathbf{v}$ to block ℓ's cache
4 read the cache with $\mathbf{q}$, one row of attention
5 **end for**
6 sample the next token from the logits

a state that grows: 2LdT numbers; Llama 7B at 4,096 tokens holds 2.1 GB per sequence, beside 13.5 GB of weights



with a cache

Figure 44b: with one, each step computes one new column

Every Weight, and What It Costs

Shrinking the cache

share keys and values among heads: the largest Llama 2 has 64 query heads and 8 key–value heads, eight times smaller

compress, window, or recur: a latent vector per token (DeepSeek); the last W tokens; a fixed state (§11)

one to two orders of magnitude: Llama 3.1 405B at 131,072 tokens keeps 68 GB instead of 1.1 TB

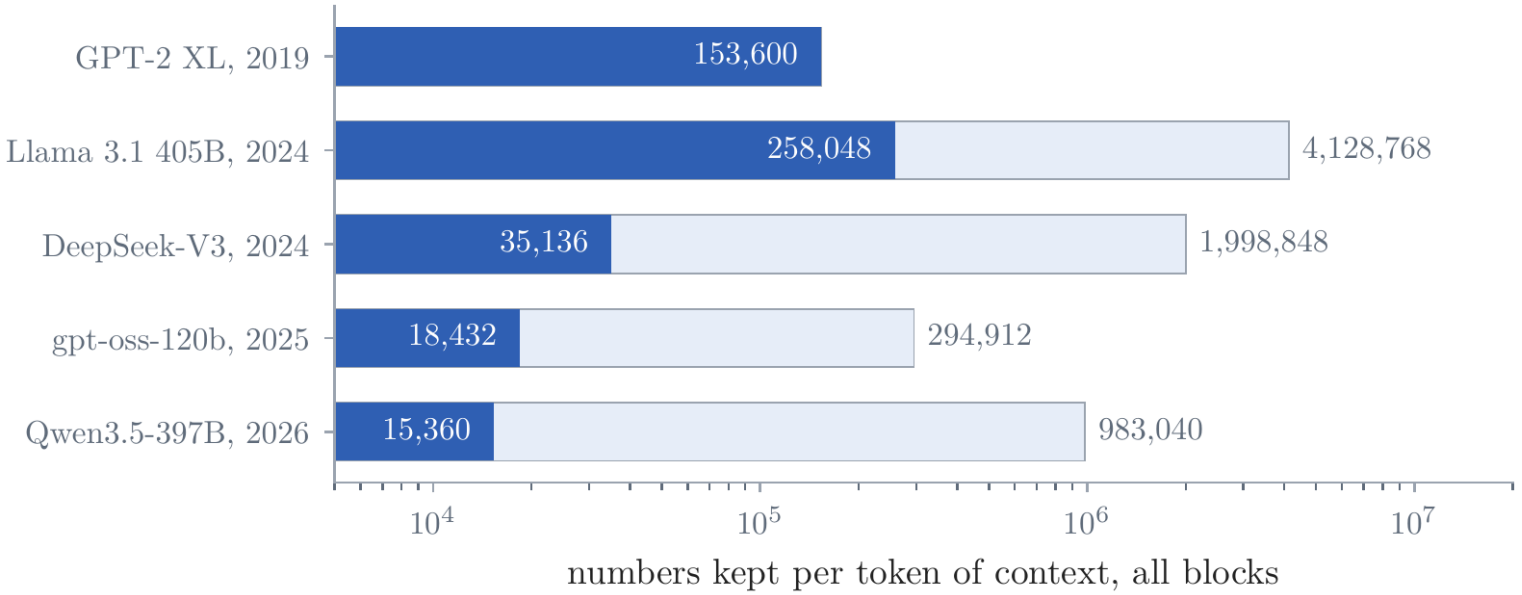


Figure 45: numbers cached per token; light bars, full multi-head attention

Strengths, Limitations, and Variants

Strengths

parallel training: one pass computes every prediction of a text

any position reads any other in one step: copying and retrieval come easily (Jelassi et al., 2024)

the context is the memory: every token keeps its own vector; no fixed-size summary

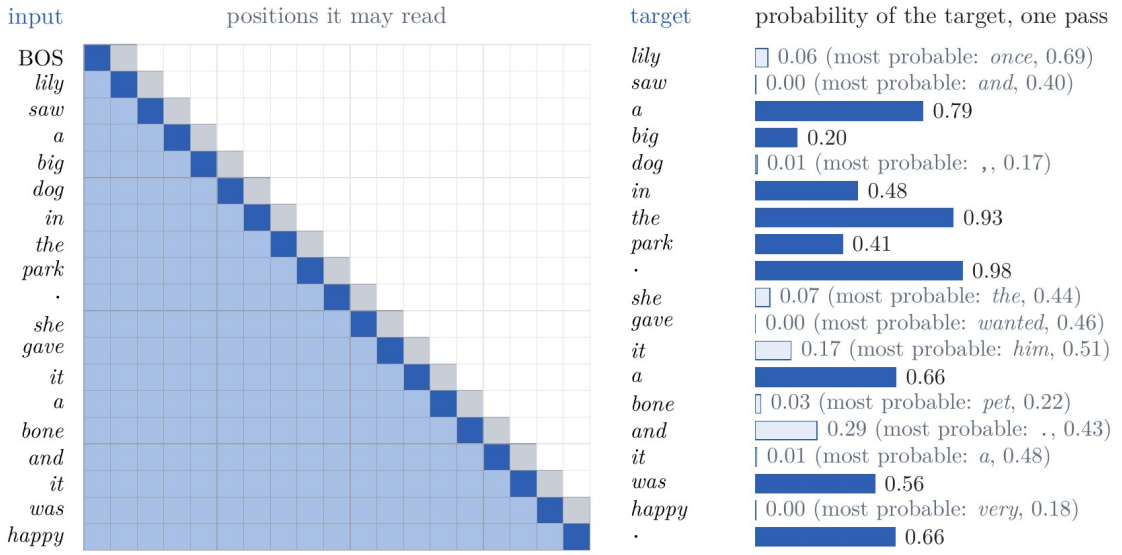


Figure 35: one pass computes every prediction; each row reads every earlier position

Strengths, Limitations, and Variants

Limitations

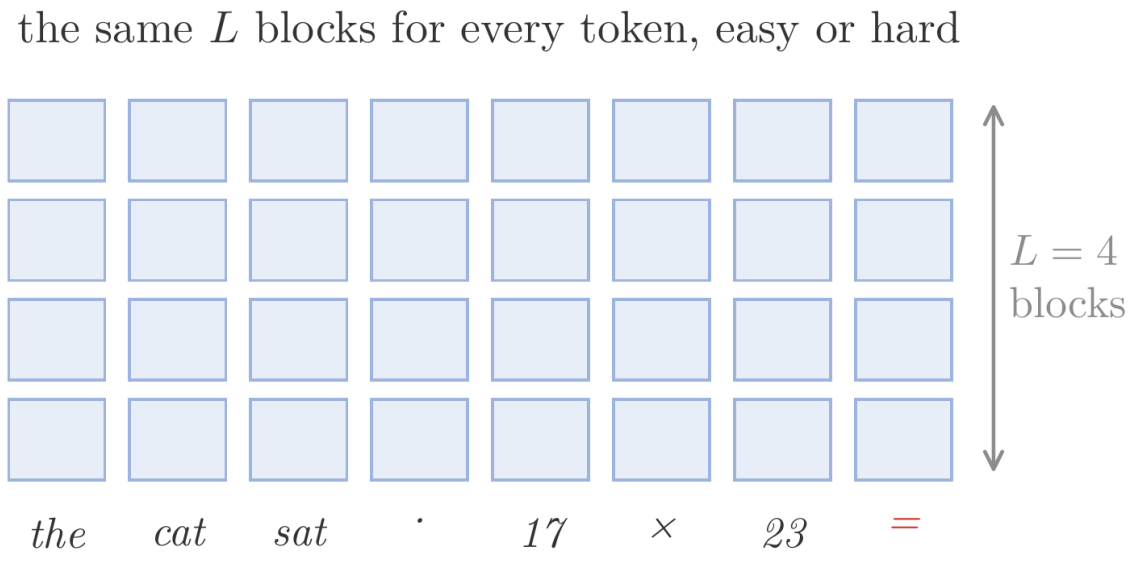
long contexts are costly: work grows with T^2, the cache with T

a fixed computation per token: L blocks for every token, easy or hard; intermediate tokens escape it

lengths beyond training: unseen angles or offsets; trained on short examples, models often fail on long ones (Anil et al., 2022)

using a long context well: documents in the middle are used worst (Liu et al., 2024)

no memory beyond the context: the weights are fixed after training



Slide figure: one pass, the same depth for every token

Strengths, Limitations, and Variants

Variants

the problem: the cost grows with the context, and the depth is fixed → **the fix here**: a state of fixed size, or a loop

a state of fixed size: replace the exponential by a product, and attention becomes a recurrence

a bottleneck again: so hybrids, Jamba with 1 attention block in 8 and Qwen3.5 with 1 in 4

a loop: one block applied several times; k layers looped L times nearly match kL layers (Saunshi et al., 2025)

open: loops, recurrence, or written reasoning; which spends computation at test time best?

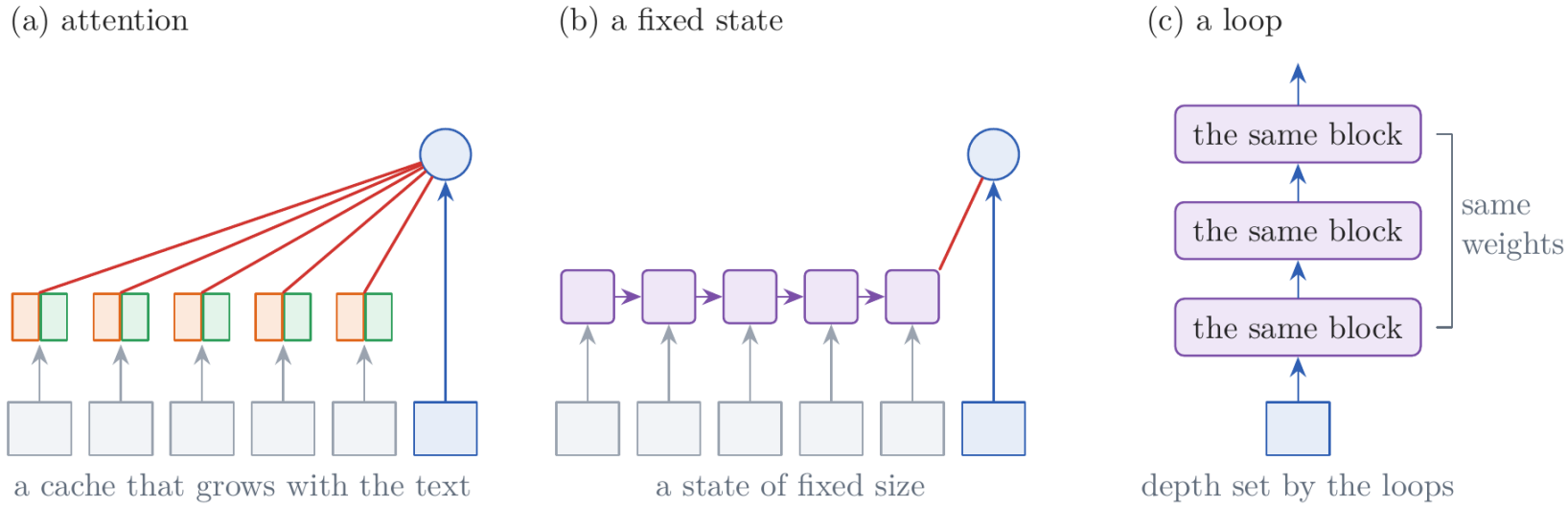


Figure 46: (a) attention, (b) a fixed state, (c) a loop

What Carries Forward

From a toy translator to a language model

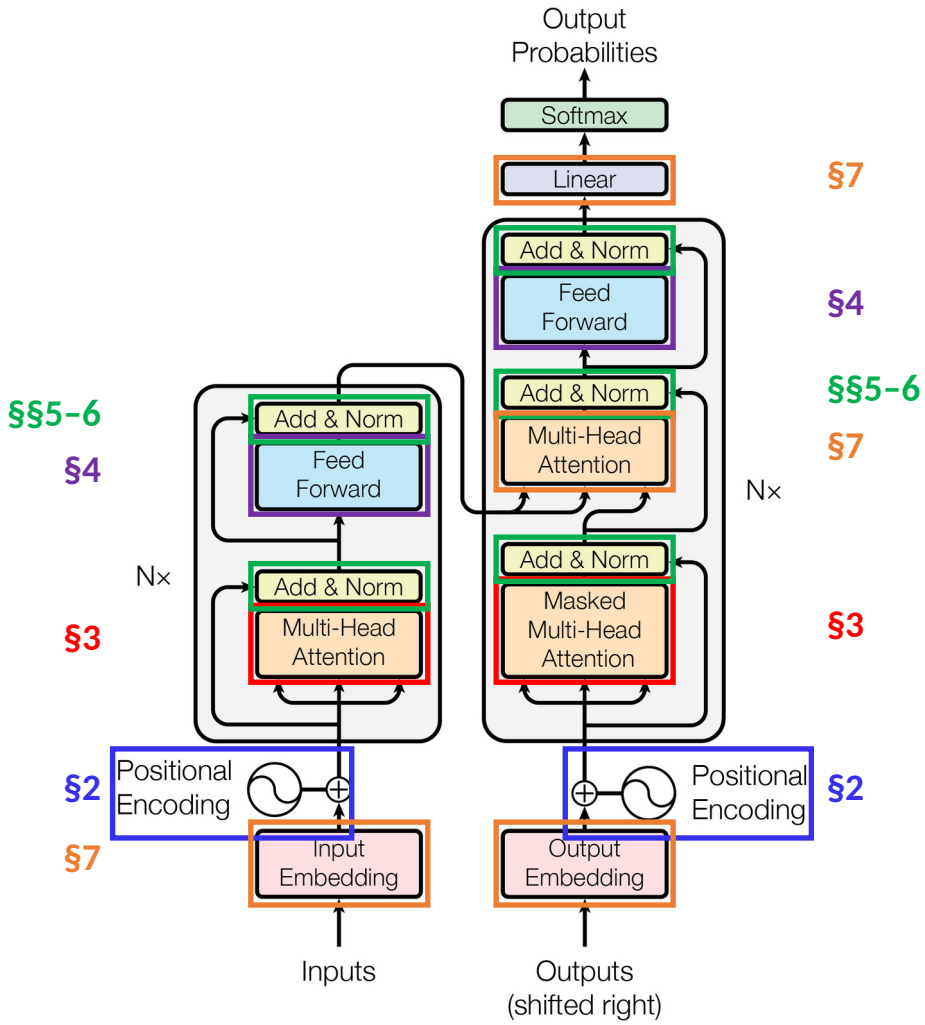
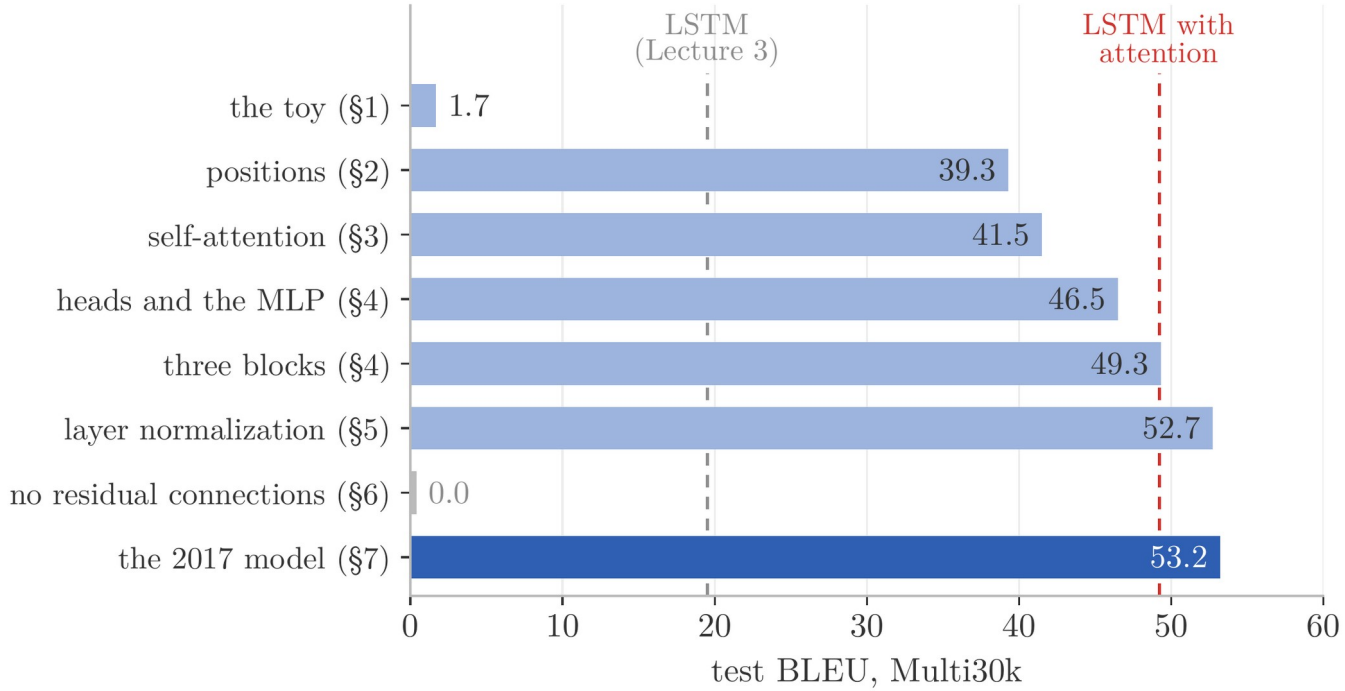


Figure 2: Vaswani et al. (2017), Fig. 1



Slide figure: the build of Part 1, test BLEU on Multi30k

the decoder alone is a language model: two tests check its mask

what it costs: about $12Ld^2$ parameters and 6N FLOPs per token; attention's work grows with T^2, the cache with T

next week, a base model: tokens, data, the training recipe, scaling laws ($C \approx 6ND$); then a chat assistant, by supervised fine-tuning